ĐẠI HỌC THÁI NGUYÊN
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO DỰ ÁN
HỌC PHẦN HỌC SÂU (DEEP LEARNING)

Đề tài:

PHÁT HIỆN VÀ ĐỊNH VỊ BẤT THƯỜNG TRÊN SẢN PHẨM CÔNG NGHIỆP BẰNG HỌC KHÔNG GIÁM SÁT

Tên nhóm: **NHÓM 13**

Sinh viên thực hiện: **Đặng Hoàng An**
Trần Văn Dưỡng
Lớp KHMT K22A

Giáo viên hướng dẫn: TS. Ngô Hữu Huy

THÁI NGUYÊN, NĂM 2026

MỤC LỤC

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT 2
1.1 Trí tuệ nhân tạo, học máy và học sâu 2
1.1.1 Trí tuệ nhân tạo 2
1.1.2 Học máy 2
1.1.3 Học sâu 3
1.1.4 Quan hệ giữa ba khái niệm 4
1.2 Dữ liệu và tiền xử lý 4
1.3 Mạng nơ-ron và quá trình huấn luyện 5
1.3.1 Cấu trúc mạng nơ-ron 5
1.3.2 Quá trình huấn luyện 6
1.3.3 Quá khớp và cách hạn chế 6
1.4 Mạng nơ-ron tích chập 7
1.5 Đặc trưng học trước 7
1.6 Thị giác máy tính và ứng dụng 8
1.7 Bài toán phát hiện bất thường không giám sát 9
1.7.1 Kiểm tra lỗi sản phẩm công nghiệp và các khó khăn 9
1.7.2 Phát hiện và định vị bất thường 9
1.7.3 Vì sao chỉ học từ ảnh tốt 10
1.7.4 Hai loại bất thường 10
1.7.5 Các hướng tiếp cận và vị trí của ba phương pháp 10
1.8 Đánh giá mô hình 11
1.8.1 Các chỉ số phân loại thông thường 11
1.8.2 Đường cong ROC và AUROC 11
1.9 Công cụ và thư viện 12

CHƯƠNG 2. PHƯƠNG PHÁP PHÁT HIỆN BẤT THƯỜNG 13
2.1 Phát biểu bài toán 13
2.2 Tổng quan giải pháp 13
2.3 Bộ dữ liệu MVTec AD 14
2.4 Tiền xử lý và giao thức chung 14
2.5 Lựa chọn phương pháp 17
2.6 Autoencoder 18
2.6.1 Nguyên lý 18
2.6.2 Kiến trúc 18

2.6.3 Huấn luyện . . . 20
2.6.4 Bản đồ và điểm bất thường . . . 20
2.6.5 Ưu điểm và hạn chế . . . 20
2.7 PaDiM . . . 20
2.7.1 Nguyên lý . . . 20
2.7.2 Trích xuất đặc trưng . . . 21
2.7.3 Ước lượng phân phối và tính điểm . . . 21
2.7.4 Triển khai và bộ nhớ . . . 22
2.7.5 Ưu điểm và hạn chế . . . 22
2.8 PatchCore . . . 22
2.8.1 Nguyên lý . . . 22
2.8.2 Đặc trưng patch có ngữ cảnh lân cận . . . 22
2.8.3 Bộ nhớ và chọn coreset . . . 22
2.8.4 Tính điểm . . . 23
2.8.5 Ưu điểm và hạn chế . . . 24
2.9 Tiêu chí so sánh và quy trình thực nghiệm . . . 24
2.10 Cài đặt và chạy thực nghiệm . . . 25
2.10.1 Tổ chức mã nguồn . . . 25
2.10.2 Quy trình chạy một loại sản phẩm . . . 25
2.10.3 Cách chạy trên Colab . . . 25
2.10.4 Các vấn đề kỹ thuật đã xử lý . . . 26
2.10.5 Đo thời gian và vẽ hình minh họa . . . 26

CHƯƠNG 3. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ . . . 27
3.1 Thiết lập thực nghiệm . . . 27
3.2 Kết quả tổng hợp . . . 27
3.3 Kết quả theo từng loại . . . 28
3.4 Phân tích định tính . . . 29
3.5 Chi phí tính toán . . . 29
3.6 So sánh với kết quả đã công bố . . . 32
3.7 Hạn chế . . . 32

KẾT LUẬN . . . 33

TÀI LIỆU THAM KHẢO . . . 34

PHỤ LỤC. MÃ NGUỒN VÀ CÁCH TÁI LẬP . . . 37

DANH MỤC HÌNH

Hình 1.1 Phân loại thuật toán học máy theo việc dữ liệu có nhãn hay không 3
Hình 1.2 Học sâu: mạng nơ-ron nhiều lớp tự học đặc trưng từ dữ liệu 3
Hình 1.3 Quan hệ giữa trí tuệ nhân tạo, học máy và học sâu 4
Hình 1.4 Cấu trúc mạng nơ-ron nhiều lớp 5
Hình 1.5 Các tầng của Wide-ResNet-50-2 và tầng đặc trưng mà PaDiM, PatchCore sử dụng (kích thước minh họa với ảnh 224×224; với ảnh 256×256 của đề tài là 64, 32, 16 và 8) 8
Hình 1.6 Phát hiện đối tượng, một nhiệm vụ của thị giác máy tính 9
Hình 1.7 Đường cong ROC và AUROC (hình minh họa khái niệm, không phải kết quả thực nghiệm) 12

Hình 2.1 Thiết lập bài toán: học từ ảnh tốt, mỗi ảnh kiểm thử cho điểm ảnh và bản đồ bất thường 13
Hình 2.2 Ba phương pháp theo không gian biểu diễn, cách mô tả "bình thường" và cách tính điểm 18
Hình 2.3 Autoencoder và cách tính bản đồ bất thường (đề tài dùng sai số bình phương, không dùng SSIM) 19
Hình 2.4 Quy trình PaDiM (với ảnh 256×256 của đề tài, lưới là 64×64 và $d = 100$) 21
Hình 2.5 Quy trình PatchCore (đề tài giữ coreset 10%) 23

Hình 3.1 Hazelnut, lỗi nứt. Từ trên xuống: Autoencoder, PaDiM, PatchCore 30
Hình 3.2 Capsule, lỗi nứt nhỏ. Từ trên xuống: Autoencoder, PaDiM, PatchCore . . 31

DANH MỤC BẢNG

Bảng 1.1 Các hướng tiếp cận phát hiện bất thường không giám sát trên ảnh . . . 11
Bảng 1.2 Công cụ và thư viện sử dụng trong đề tài 12

Bảng 2.1 Các lựa chọn cấu hình và lý do 15
Bảng 2.2 Số ảnh và các kiểu lỗi của MVTec AD theo loại 16
Bảng 2.3 Kiến trúc Autoencoder (mỗi lớp tích chập dùng bộ lọc 4×4, kèm chuẩn hóa theo lô và LeakyReLU, trừ lớp ra) 19
Bảng 2.4 Tiêu chí so sánh và cách đo 24
Bảng 2.5 Cấu hình đã chạy của ba phương pháp 25

Bảng 3.1 Môi trường chạy thực nghiệm 27
Bảng 3.2 Các thiết lập dùng chung cho mọi lần chạy 27
Bảng 3.3 AUROC trung bình theo nhóm loại (cao nhất in đậm) 28
Bảng 3.4 AUROC theo từng loại (AE: Autoencoder; cao nhất mỗi chỉ số in đậm) 28
Bảng 3.5 Chi phí tính toán trên Tesla T4 30

PHÂN CÔNG NHIỆM VỤ

Thành viên	Nhiệm vụ
Đặng Hoàng An	Xây dựng và huấn luyện Autoencoder; chạy PatchCore trên 8 loại (bottle, cable, capsule, carpet, grid, hazelnut, leather, metal_nut); demo; tài liệu hướng dẫn của kho mã nguồn.
Trần Văn Dưỡng	Module đọc dữ liệu MVTec AD; module đánh giá (AUROC, hình minh họa, ghi kết quả); PaDiM; chạy PatchCore trên 7 loại (pill, screw, tile, toothbrush, transistor, wood, zipper).
Cả nhóm	Cấu hình và giao thức đánh giá chung; tiện ích chạy Anomalib; tổng hợp kết quả; viết báo cáo.

MỞ ĐẦU

Kiểm tra chất lượng là khâu bắt buộc trong sản xuất công nghiệp: sản phẩm có vết xước, nứt, nhiễm bẩn hay thiếu chi tiết phải bị loại trước khi đến tay người dùng. Kiểm tra bằng mắt người chậm, phụ thuộc kinh nghiệm và dễ bỏ sót khi sản lượng lớn hoặc lỗi nhỏ. Thị giác máy tính có thể tự động hóa khâu này, nhưng gặp một khó khăn về dữ liệu: ảnh sản phẩm lỗi hiếm, đa dạng và khó lường trước, còn việc khoanh vùng lỗi trên từng ảnh để làm nhãn tốn nhiều công sức, trong khi mô hình phân loại có giám sát chỉ nhận ra những kiểu lỗi đã thấy. Ngược lại, ảnh sản phẩm tốt có sẵn với số lượng lớn. Vì vậy, hướng học không giám sát, tức chỉ học từ ảnh tốt rồi coi mọi sai khác đáng kể là bất thường, phù hợp hơn với điều kiện thực tế [1].

Đề tài xây dựng và đánh giá ba phương pháp phát hiện và định vị bất thường không giám sát trên ảnh sản phẩm công nghiệp, nhằm trả lời ba câu hỏi: phương pháp nào phát hiện và khoanh vùng lỗi chính xác nhất; phương pháp nào phù hợp với loại sản phẩm nào (bề mặt hay vật thể, vị trí cố định hay xoay); và mỗi phương pháp đánh đổi giữa độ chính xác với thời gian, bộ nhớ ra sao.

Ba phương pháp được chọn đại diện cho hai hướng tiếp cận. Autoencoder, do nhóm tự xây dựng, học tái tạo ảnh từ đầu và dùng sai số tái tạo làm tín hiệu bất thường. PaDiM [2] và PatchCore [3] dùng đặc trưng của mạng tích chập đã học trước trên ImageNet, lần lượt mô hình hóa đặc trưng bình thường bằng phân phối Gauss theo vị trí và bằng một bộ nhớ các vector đặc trưng. Dữ liệu là đủ 15 loại sản phẩm của bộ MVTec AD [1]. Đề tài dừng ở mức đánh giá trên dữ liệu ảnh có sẵn, chưa triển khai trên dây chuyền và chưa chọn ngưỡng quyết định cho sử dụng thực tế.

Về cách làm, nhóm tìm hiểu cơ chế của từng phương pháp qua bài báo gốc, cài đặt chúng theo cùng một giao thức, chạy trên cả 15 loại và đánh giá bằng image-AUROC cho nhiệm vụ phát hiện ảnh lỗi, pixel-AUROC cho nhiệm vụ định vị vùng lỗi. Kết quả cho thấy PatchCore đạt image-AUROC trung bình 0,983 và pixel-AUROC 0,980, sát số liệu công bố; PaDiM đạt 0,890 và 0,960 với thời gian xây dựng mô hình ngắn nhất; Autoencoder đạt 0,756 và 0,843. Như vậy, với vài trăm ảnh tốt cho mỗi loại, dùng lại đặc trưng học trước hiệu quả hơn nhiều so với học tái tạo ảnh từ đầu.

Chương 1 trình bày cơ sở lý thuyết: học sâu, mạng tích chập, đặc trưng học trước, bài toán phát hiện bất thường và cách đánh giá. Chương 2 trình bày bộ dữ liệu, giao thức đánh giá và cách triển khai ba phương pháp. Chương 3 trình bày kết quả thực nghiệm và đánh giá.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

Chương này đặt đề tài vào bức tranh chung của trí tuệ nhân tạo và học máy, rồi trình bày các nền tảng mà ba phương pháp dựa vào: dữ liệu và tiền xử lý, huấn luyện mạng nơ-ron (Autoencoder), mạng tích chập (cả ba phương pháp), đặc trưng học trước (PaDiM, PatchCore), thị giác máy tính, bài toán phát hiện bất thường, cách đánh giá và các thư viện được dùng.

1.1. Trí tuệ nhân tạo, học máy và học sâu

1.1.1. Trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence, AI) là lĩnh vực xây dựng các hệ thống máy tính thực hiện được những việc vốn cần trí tuệ con người, như nhận biết, suy luận, học từ kinh nghiệm và ra quyết định. AI thường được chia làm ba cấp độ. AI hẹp (narrow AI) giải quyết một nhiệm vụ cụ thể, như nhận dạng khuôn mặt hay gợi ý sản phẩm. AI tổng quát (general AI) học và làm được mọi nhiệm vụ trí tuệ như con người. AI siêu việt (super AI) vượt con người ở mọi lĩnh vực. Hai cấp độ sau hiện vẫn ở mức lý thuyết; mọi hệ thống đang dùng trong thực tế, kể cả hệ thống kiểm tra lỗi sản phẩm của đề tài, đều là AI hẹp.

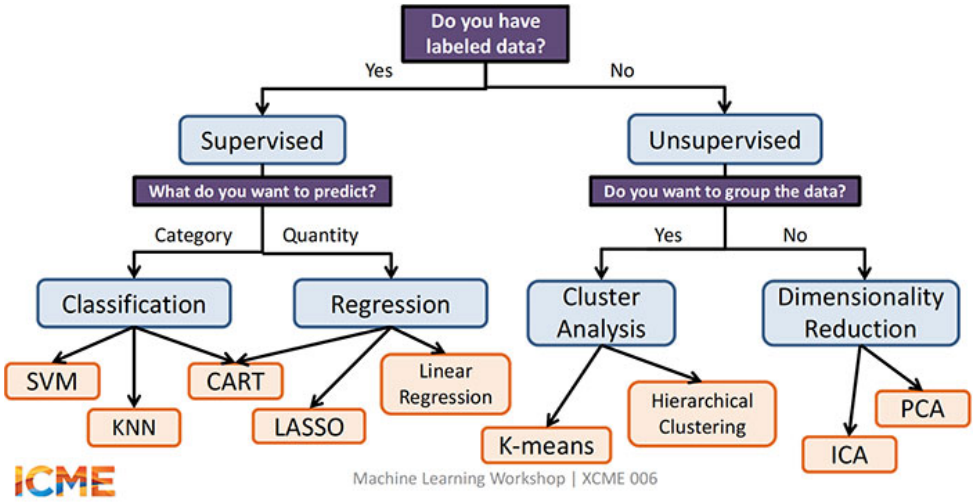
1.1.2. Học máy

Học máy (Machine Learning, ML) là một nhánh của AI, trong đó hệ thống tự rút ra quy luật từ dữ liệu thay vì được lập trình sẵn từng quy tắc. Mô hình được xây dựng trên dữ liệu huấn luyện, rồi được dùng để dự đoán hoặc ra quyết định cho dữ liệu mới mà nó chưa thấy. Vì vậy, điều quan trọng của một mô hình học máy không phải là thuộc lòng dữ liệu huấn luyện mà là tổng quát hóa được sang dữ liệu mới [4].

Theo cách dùng dữ liệu, học máy chia làm ba nhóm (Hình 1.1):

- **Học có giám sát:** dữ liệu đã gán nhãn, mô hình học cách dự đoán nhãn, ví dụ phân loại ảnh sản phẩm thành "tốt" hoặc "lỗi" khi có sẵn nhiều ảnh lỗi đã gán nhãn.
- **Học không giám sát:** dữ liệu không có nhãn, mô hình tìm cấu trúc ẩn trong dữ liệu, như nhóm các mẫu giống nhau, giảm số chiều hoặc nhận ra mẫu nào khác thường.
- **Học tăng cường:** hệ thống học qua tương tác với môi trường và nhận thưởng hoặc phạt cho từng hành động; nhóm này không liên quan đến đề tài.

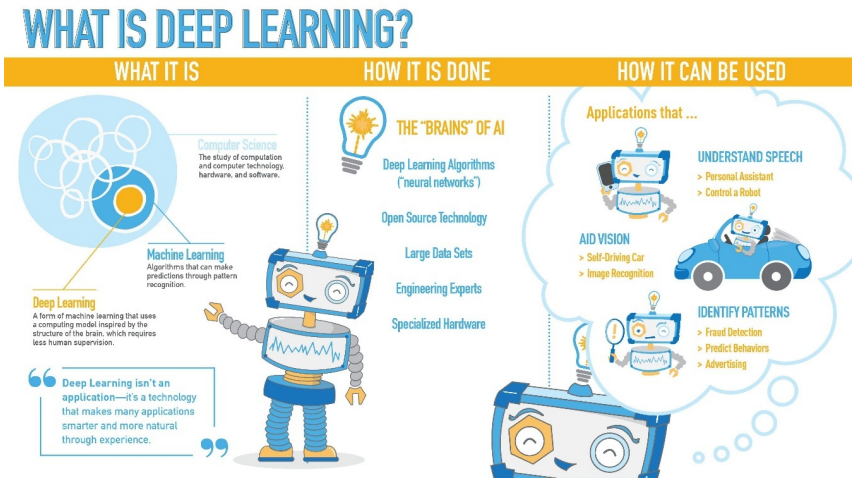
Phát hiện bất thường trong đề tài thuộc học không giám sát theo nghĩa hẹp: mô hình chỉ được thấy ảnh tốt, không thấy ảnh lỗi hay nhãn vùng lỗi nào khi học. Một số tài liệu gọi thiết lập này là học một lớp, vì dữ liệu huấn luyện chỉ thuộc một lớp là "bình thường". Mục 1.7 trình bày chi tiết bài toán này.



Hình 1.1. Phân loại thuật toán học máy theo việc dữ liệu có nhãn hay không

1.1.3. Học sâu

Học sâu (Deep Learning, DL) là một nhánh của học máy, dùng mạng nơ-ron nhiều lớp để học cách biểu diễn dữ liệu. Điểm khác biệt chính so với học máy truyền thống là đặc trưng không do con người thiết kế mà được mạng tự học từ dữ liệu, theo thứ bậc: với ảnh, các lớp đầu học cạnh và kết cấu, các lớp sau ghép chúng thành bộ phận và vật thể [5]. Học sâu phát triển mạnh nhờ ba yếu tố: lượng dữ liệu lớn, sức tính của GPU, và các thư viện như PyTorch giúp xây dựng và huấn luyện mạng dễ dàng hơn (Hình 1.2).



Hình 1.2. Học sâu: mạng nơ-ron nhiều lớp tự học đặc trưng từ dữ liệu

Một số kiến trúc học sâu tiêu biểu và mức liên quan đến đề tài:

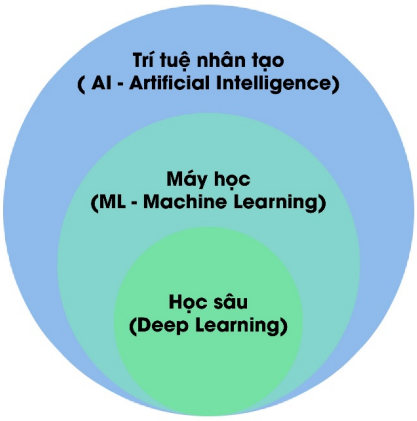
- **Mạng nơ-ron tích chập (CNN):** xử lý ảnh, giữ được cấu trúc không gian; là nền của cả ba phương pháp trong đề tài (Mục 1.4).
- **Autoencoder:** mạng nén rồi dựng lại dữ liệu, dùng để giảm chiều hoặc phát hiện bất thường; là một trong ba phương pháp của đề tài.
- **Mạng hồi quy (RNN, LSTM):** xử lý dữ liệu chuỗi như văn bản, tiếng nói; không dùng

trong đề tài.

- **Transformer:** dùng cơ chế chú ý để mô hình hóa quan hệ giữa các phần của dữ liệu, phổ biến trong xử lý ngôn ngữ và ngày càng dùng nhiều cho ảnh; không dùng trong đề tài.

1.1.4. Quan hệ giữa ba khái niệm

Ba khái niệm lồng nhau: học sâu là một phần của học máy, học máy là một phần của trí tuệ nhân tạo (Hình 1.3). Cả ba phương pháp của đề tài đều thuộc học sâu: Autoencoder là một mạng nơ-ron tự huấn luyện, còn PaDiM và PatchCore dựa trên đặc trưng của một mạng nơ-ron sâu đã huấn luyện sẵn.



Hình 1.3. Quan hệ giữa trí tuệ nhân tạo, học máy và học sâu

1.2. Dữ liệu và tiền xử lý

Dữ liệu quyết định chất lượng của mô hình học máy. Dữ liệu thường được chia làm ba loại: có cấu trúc (bảng với các cột rõ ràng, như cơ sở dữ liệu), phi cấu trúc (ảnh, âm thanh, văn bản) và bán cấu trúc (JSON, XML, nhật ký hệ thống). Ảnh sản phẩm của đề tài là dữ liệu phi cấu trúc: mỗi ảnh là một ma trận giá trị điểm ảnh, ý nghĩa nằm ở cách các điểm ảnh sắp xếp chứ không ở từng giá trị riêng lẻ. Đây là loại dữ liệu mà học sâu xử lý tốt hơn hẳn các phương pháp dùng đặc trưng thủ công.

Trước khi đưa vào mô hình, dữ liệu cần được tiền xử lý. Các bước thường gặp và cách đề tài áp dụng:

- **Làm sạch:** loại mẫu hỏng, thiếu hoặc sai. MVTec AD là bộ dữ liệu đã được làm sạch nên đề tài chỉ kiểm tra đủ số ảnh.
- **Chuẩn hóa:** đưa dữ liệu về cùng thang đo, như đổi ảnh về cùng kích thước và đưa giá trị điểm ảnh về cùng phân phối. Với mạng học trước, dữ liệu phải được chuẩn hóa giống lúc mạng đó được huấn luyện.

- **Chia tập:** tách tập huấn luyện để xây dựng mô hình và tập kiểm thử để đánh giá. Tập kiểm thử không được dùng khi xây dựng hay chọn mô hình, nếu không kết quả đánh giá sẽ lạc quan hơn thực tế.
- **Tăng cường dữ liệu:** tạo thêm mẫu bằng cách xoay, lật, đổi độ sáng. Kỹ thuật này hữu ích khi phép biến đổi không làm thay đổi bản chất mẫu; với bài toán tìm lỗi, nó có thể biến ảnh tốt thành ảnh trông giống lỗi, nên đề tài không dùng (Mục 2.4).

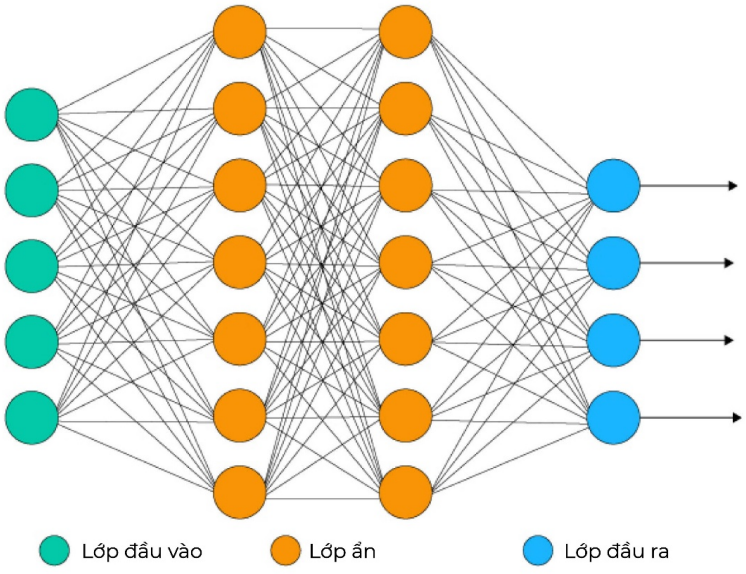
1.3. Mạng nơ-ron và quá trình huấn luyện

1.3.1. Cấu trúc mạng nơ-ron

Một nơ-ron nhân tạo nhận vector đầu vào x, tính tổ hợp tuyến tính với trọng số w và độ lệch b, rồi đưa qua hàm kích hoạt phi tuyến φ:

$$y = \varphi\big(w^{\top} x + b\big) \tag{1.1}$$

Nhiều nơ-ron xếp thành lớp, nhiều lớp nối tiếp thành mạng; đầu ra của lớp trước là đầu vào của lớp sau (Hình 1.4) [6]. Hàm kích hoạt phi tuyến là điều kiện để mạng nhiều lớp biểu diễn được quan hệ phi tuyến; nếu bỏ đi, cả mạng chỉ tương đương một phép biến đổi tuyến tính. Autoencoder của đề tài dùng LeakyReLU, $\varphi(t) = \max(t, 0{,}2\,t)$, ở các lớp ẩn và hàm sigmoid ở lớp ra để giá trị điểm ảnh tái tạo nằm trong $[0, 1]$.



Hình 1.4. Cấu trúc mạng nơ-ron nhiều lớp

Mạng gồm lớp vào nhận dữ liệu thô, các lớp ẩn biến đổi dữ liệu thành đặc trưng ngày càng trừu tượng, và lớp ra cho kết quả cuối cùng. Mạng có nhiều lớp ẩn được gọi là mạng sâu; càng nhiều lớp, mạng càng biểu diễn được quan hệ phức tạp, nhưng cũng càng khó huấn luyện và càng cần nhiều dữ liệu.

1.3.2. Quá trình huấn luyện

Huấn luyện là tìm tham số θ (toàn bộ trọng số và độ lệch) làm nhỏ hàm mất mát $L(\theta)$ trên dữ liệu huấn luyện [7]. Một vòng huấn luyện gồm các bước:

1. **Khởi tạo trọng số** ngẫu nhiên theo phân phối phù hợp, để các nơ-ron không học giống hệt nhau.
2. **Lan truyền tiến:** đưa một lô dữ liệu qua mạng để được đầu ra dự đoán.
3. **Tính mất mát:** đo sai lệch giữa dự đoán và mục tiêu, ví dụ sai số bình phương trung bình (MSE) cho bài toán dựng lại ảnh hoặc entropy chéo cho bài toán phân loại.
4. **Lan truyền ngược:** tính gradient $\nabla_\theta L$ theo quy tắc đạo hàm hàm hợp, đi từ lớp ra về lớp vào [5].
5. **Cập nhật tham số** ngược hướng gradient bằng một thuật toán tối ưu, rồi lặp lại với lô tiếp theo.

Mỗi lần cập nhật dùng một lô nhỏ (mini-batch) thay vì toàn bộ dữ liệu; một lượt đi qua hết dữ liệu gọi là một epoch. Thuật toán tối ưu đơn giản nhất là SGD, cập nhật mọi tham số với cùng một bước. Đề tài dùng Adam [8], trong đó mỗi tham số có bước cập nhật riêng, điều chỉnh theo trung bình động của gradient và của bình phương gradient, nên thường hội tụ nhanh và ít phải chỉnh tốc độ học.

1.3.3. Quá khớp và cách hạn chế

Quá khớp (overfitting) là khi mô hình học thuộc cả nhiễu và chi tiết riêng của dữ liệu huấn luyện: mất mát trên tập huấn luyện rất thấp nhưng kết quả trên dữ liệu mới kém [4]. Nguy cơ này cao khi mô hình có nhiều tham số mà dữ liệu ít, đúng tình huống của đề tài: mỗi loại sản phẩm chỉ có 60 đến 391 ảnh tốt. Các cách hạn chế thường dùng:

- **Chính quy hóa L2 (weight decay):** cộng vào mất mát một khoản tỷ lệ với bình phương độ lớn trọng số, để trọng số không quá lớn. Autoencoder của đề tài dùng weight decay 10^{-5}.
- **Dropout:** tắt ngẫu nhiên một phần nơ-ron khi huấn luyện để mạng không phụ thuộc vào vài nơ-ron.
- **Dừng sớm (early stopping):** ngừng huấn luyện khi kết quả trên tập kiểm định thôi cải thiện.
- **Giảm số tham số cần học:** dùng mạng nhỏ hơn hoặc dùng lại đặc trưng học trước. PaDiM và PatchCore không học tham số mạng nào trên MVTec AD, nên gần như không có nguy cơ quá khớp theo nghĩa này.

Với Autoencoder, quá khớp còn có một dạng riêng: mạng quá mạnh học được cách dựng lại mọi thứ, kể cả lỗi, nên mất khả năng phát hiện. Cổ chai hẹp (Mục 2.5) là cách đề tài hạn chế điều này.

1.4. Mạng nơ-ron tích chập

Với ảnh, lớp kết nối đầy đủ có hai nhược điểm: số tham số rất lớn (ảnh $256 \times 256 \times 3$ có gần 200 nghìn giá trị đầu vào) và mất thông tin điểm ảnh nào nằm cạnh điểm ảnh nào. Lớp tích chập khắc phục cả hai bằng một bộ lọc nhỏ K trượt trên ảnh [9]:

$$h(u, v) = \sum_{i} \sum_{j} \sum_{c} K(i, j, c)\, x(u + i,\ v + j,\ c) + b \tag{1.2}$$

Cùng một bộ lọc được dùng ở mọi vị trí (chia sẻ trọng số), nên số tham số chỉ phụ thuộc kích thước bộ lọc và số kênh, không phụ thuộc kích thước ảnh. Đầu ra vẫn là một bản đồ có tọa độ không gian, nên mỗi vị trí của bản đồ đặc trưng ứng với một vùng (patch) của ảnh gốc. Tính chất này là cơ sở để cả ba phương pháp trong đề tài tạo được bản đồ bất thường theo vị trí.

Bước trượt (stride) lớn hơn 1 làm bản đồ đặc trưng nhỏ dần; tích chập chuyển vị (transposed convolution) làm điều ngược lại, phóng to bản đồ. Autoencoder dùng cặp phép toán này để nén rồi dựng lại ảnh. Chuẩn hóa theo lô (batch normalization) đưa đầu ra mỗi lớp về trung bình 0, phương sai 1 trên từng lô, giúp huấn luyện ổn định hơn [10].

Khi xếp chồng nhiều lớp, mỗi vị trí ở lớp sâu nhìn được một vùng ảnh rộng hơn (trường tiếp nhận lớn hơn). Các lớp nông phản hồi với cạnh, màu và kết cấu đơn giản; các lớp sâu biểu diễn cấu trúc phức tạp hơn nhưng có độ phân giải không gian thấp hơn [11]. PaDiM và PatchCore chọn các tầng trung gian vì cần cả hai: đủ chi tiết để thấy lỗi nhỏ và đủ ngữ cảnh để phân biệt lỗi với biến thiên bình thường.

ResNet [12] cho phép huấn luyện mạng rất sâu bằng kết nối tắt: mỗi khối học phần chênh lệch $F(x)$ rồi cộng lại với đầu vào,

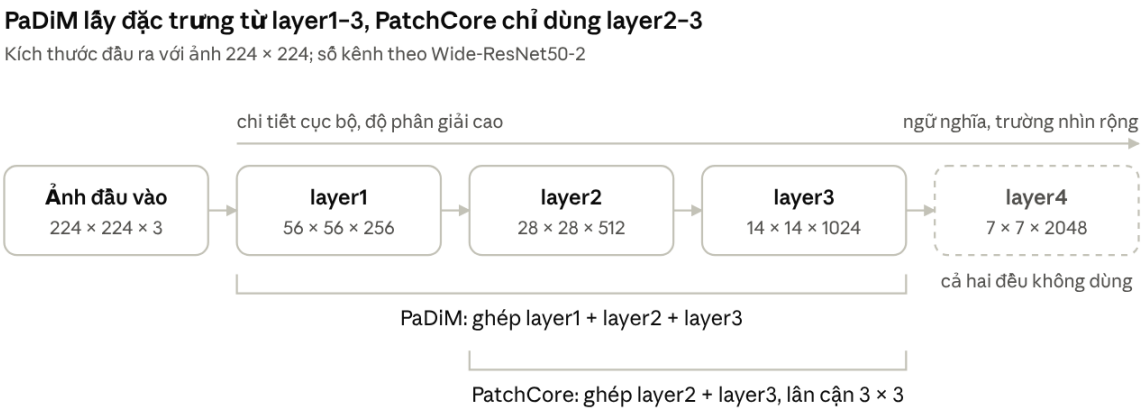
$$y = F(x) + x. \tag{1.3}$$

Các khối được nhóm thành bốn tầng `layer1`–`layer4`; mỗi tầng sau giảm một nửa kích thước không gian và tăng số kênh (Hình 1.5). Với ảnh vào 256×256 của đề tài, `layer1`, `layer2`, `layer3` cho bản đồ 64×64, 32×32, 16×16. Wide ResNet [13] giữ cấu trúc đó nhưng tăng gấp đôi số kênh bên trong mỗi khối. Đề tài dùng ResNet-18 (khoảng 11,7 triệu tham số) cho PaDiM và Wide-ResNet-50-2 (khoảng 68,9 triệu tham số) cho PatchCore. Cả hai phương pháp đều bỏ `layer4`: tầng này có độ phân giải quá thấp để định vị lỗi và mang đặc trưng thiên về nhiệm vụ phân loại ImageNet.

1.5. Đặc trưng học trước

ImageNet là bộ dữ liệu khoảng 1,2 triệu ảnh thuộc 1000 lớp đối tượng [14]. Một mạng tích chập huấn luyện để phân loại ImageNet học được các bộ lọc cho cạnh, kết cấu và hình dạng; các bộ lọc ở tầng nông và tầng giữa dùng lại được cho dữ liệu khác với hiệu quả tốt [11].

PaDiM và PatchCore dùng mạng như vậy làm bộ trích xuất đặc trưng cố định: không cập



Hình 1.5. Các tầng của Wide-ResNet-50-2 và tầng đặc trưng mà PaDiM, PatchCore sử dụng (kích thước minh họa với ảnh 224×224; với ảnh 256×256 của đề tài là 64, 32, 16 và 8)

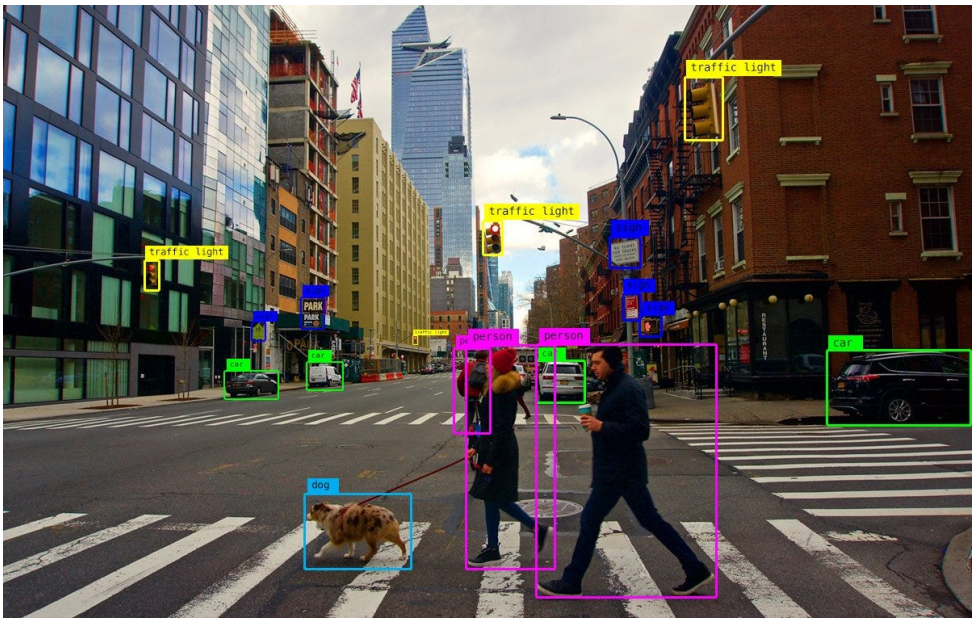
nhật trọng số nào trên MVTec AD. Phần "học" trên ảnh tốt chỉ là ghi nhận đặc trưng của chúng, dưới dạng thống kê (PaDiM) hoặc tập vector tham chiếu (PatchCore). Nhờ đó hai phương pháp không cần lan truyền ngược và không cần nhiều ảnh tốt. Đổi lại, chất lượng phụ thuộc vào việc đặc trưng ImageNet có phân biệt được lỗi công nghiệp hay không.

1.6. Thị giác máy tính và ứng dụng

Thị giác máy tính là lĩnh vực của AI giúp máy tính hiểu nội dung ảnh và video. Ba nhiệm vụ cơ bản xếp theo mức chi tiết tăng dần: phân loại ảnh (ảnh thuộc lớp nào), phát hiện đối tượng (đối tượng nằm ở hộp nào trong ảnh, Hình 1.6) và phân đoạn (từng điểm ảnh thuộc đối tượng nào). Bài toán của đề tài ghép hai mức: phát hiện ảnh lỗi tương ứng phân loại ảnh, còn định vị vùng lỗi tương ứng phân đoạn ở mức điểm ảnh. Từ khi mạng tích chập ra đời, cả ba nhiệm vụ đều do học sâu đảm nhận là chính.

Hai nhóm ứng dụng gần với đề tài nhất:

- **Kiểm tra chất lượng trong sản xuất:** camera chụp từng sản phẩm trên dây chuyền, mô hình quyết định đạt hay không đạt và chỉ ra vị trí lỗi để truy nguyên nhân. Khác với nhiều ứng dụng khác, ở đây ảnh lỗi hiếm và kiểu lỗi khó lường trước, nên hướng chỉ học từ ảnh tốt được dùng rộng rãi.
- **Phân tích ảnh y học:** học sâu được dùng để phát hiện tổn thương trên ảnh X-quang, CT, MRI. Ảnh bệnh lý hiếm hơn ảnh bình thường và việc khoanh vùng tổn thương cần chuyên gia, nên lĩnh vực này cũng gặp đúng khó khăn về dữ liệu như kiểm tra chất lượng công nghiệp.



Hình 1.6. Phát hiện đối tượng, một nhiệm vụ của thị giác máy tính

1.7. Bài toán phát hiện bất thường không giám sát

1.7.1. Kiểm tra lỗi sản phẩm công nghiệp và các khó khăn

Trên dây chuyền sản xuất, mỗi sản phẩm cần được kiểm tra để loại bỏ những chiếc bị xước, nứt, nhiễm bẩn, biến dạng hay thiếu chi tiết. Kiểm tra bằng mắt người chậm hơn tốc độ dây chuyền, phụ thuộc kinh nghiệm và độ tập trung của người kiểm tra, và dễ bỏ sót lỗi nhỏ. Tự động hóa khâu này bằng thị giác máy tính gặp bốn khó khăn đặc thù:

- **Ảnh lỗi hiếm.** Dây chuyền ổn định có tỷ lệ lỗi thấp, nên thu đủ ảnh lỗi để huấn luyện tốn nhiều thời gian. Ngược lại, ảnh sản phẩm tốt có sẵn với số lượng lớn.
- **Lỗi đa dạng và khó lường trước.** Không thể liệt kê trước mọi kiểu lỗi; một kiểu lỗi mới có thể xuất hiện bất cứ lúc nào khi nguyên liệu hay máy móc thay đổi.
- **Gán nhãn tốn kém.** Khoanh vùng lỗi chính xác đến từng điểm ảnh cần người có chuyên môn và nhiều công sức.
- **Lỗi nhỏ trên nền biến thiên.** Nhiều lỗi chỉ chiếm một phần rất nhỏ diện tích ảnh, trong khi ảnh tốt vẫn khác nhau tự nhiên về ánh sáng, tư thế và kết cấu bề mặt.

1.7.2. Phát hiện và định vị bất thường

Bài toán gồm hai nhiệm vụ trả lời hai câu hỏi khác nhau. **Phát hiện** trả lời "sản phẩm này có lỗi không": đầu ra là một số cho cả ảnh, điểm bất thường $s(x)$, và được đánh giá trên từng ảnh. **Định vị** trả lời "lỗi nằm ở đâu": đầu ra là bản đồ bất thường $A_x(u, v)$ cùng kích thước ảnh, mỗi điểm ảnh một giá trị, và được đánh giá trên từng điểm ảnh so với mặt nạ vùng lỗi. Điểm ảnh thường được suy ra từ bản đồ, ví dụ $s(x) = \max_{u,v} A_x(u, v)$. Điểm càng cao thì càng khác thường; điểm không phải xác suất có lỗi và không có thang đo chung giữa các phương pháp.

Hai nhiệm vụ có thể cho kết quả khác nhau trên cùng một mô hình. Với một hạt dẻ bị nứt, mô hình có thể khoanh đúng vết nứt trên bản đồ (định vị tốt), nhưng nếu bản đồ của ảnh tốt cũng có vài điểm sáng thì điểm lớn nhất của ảnh tốt và ảnh lỗi khó phân biệt (phát hiện kém). Ngược lại, một mô hình có thể nhận ra ảnh lỗi nhờ một vùng sai lệch lớn mà vẫn khoanh lệch vị trí. Vì vậy đề tài đánh giá riêng hai nhiệm vụ (Mục 1.8). Trong ứng dụng thực tế, phát hiện quyết định loại hay giữ sản phẩm, còn định vị giúp người vận hành kiểm tra lại và truy nguyên nhân lỗi.

1.7.3. Vì sao chỉ học từ ảnh tốt

Một bộ phân loại có giám sát cần các ví dụ lỗi đã gán nhãn và có thể khó khái quát sang kiểu lỗi chưa xuất hiện trong tập huấn luyện; với ảnh lỗi hiếm và lỗi mới liên tục xuất hiện, điều kiện này khó đáp ứng. Hướng không giám sát đảo ngược cách đặt vấn đề: thay vì học "lỗi trông như thế nào", mô hình học "bình thường trông như thế nào" từ tập ảnh tốt $\mathcal{X}_{\text{train}} = \{x_1, \ldots, x_N\}$, rồi coi mọi sai khác đáng kể là bất thường [1]. Cách này chỉ cần ảnh tốt, vốn dễ thu thập, và về nguyên tắc phát hiện được cả kiểu lỗi chưa từng xuất hiện. Đổi lại, mô hình không biết đâu là lỗi "thật": mọi khác thường, kể cả một vết bẩn vô hại trên nền, đều có thể bị báo. Ảnh lỗi và mặt nạ vùng lỗi chỉ dùng ở bước đánh giá.

1.7.4. Hai loại bất thường

Trên ảnh công nghiệp, bất thường thường chia làm hai loại. Bất thường cấu trúc là thay đổi cục bộ về bề mặt hay hình dạng, như vết xước, vết nứt, lỗ thủng, vết bẩn. Bất thường logic là trường hợp từng chi tiết đều trông bình thường nhưng tổng thể sai, như thiếu linh kiện, lắp sai vị trí hay lật ngược. Phân biệt hai loại giúp giải thích vì sao một phương pháp so sánh từng vùng nhỏ có thể bỏ sót lỗi dù khoanh vùng tốt các vết trên bề mặt.

1.7.5. Các hướng tiếp cận và vị trí của ba phương pháp

Các phương pháp phát hiện bất thường không giám sát trên ảnh khác nhau ở chỗ chúng mô tả "bình thường" bằng gì và đo "khác thường" ra sao (Bảng 1.1). Đề tài chọn ba phương pháp thuộc ba hướng đầu: Autoencoder thuộc hướng tái tạo, so sánh trực tiếp trên điểm ảnh; PaDiM thuộc hướng mô hình hóa phân phối đặc trưng; PatchCore thuộc hướng bộ nhớ đặc trưng. Hai phương pháp sau cùng dựa trên đặc trưng học trước, nhưng PaDiM gắn thống kê với từng vị trí còn PatchCore so sánh không phụ thuộc vị trí. Hai hướng còn lại được nêu để thấy vị trí của ba phương pháp, không nằm trong phạm vi thực nghiệm.

Chương 2 trình bày chi tiết cách ba phương pháp được chọn mô tả trạng thái bình thường và tính điểm bất thường.

Bảng 1.1. Các hướng tiếp cận phát hiện bất thường không giám sát trên ảnh

Hướng	Mô tả bình thường bằng	Điểm bất thường	Ví dụ
Tái tạo	Mạng học dựng lại ảnh tốt	Sai khác giữa ảnh và ảnh dựng lại	Autoencoder
Phân phối đặc trưng	Phân phối xác suất của đặc trưng học trước	Độ lệch khỏi phân phối	PaDiM
Bộ nhớ đặc trưng	Tập vector đặc trưng của ảnh tốt	Khoảng cách tới mẫu gần nhất	PatchCore
Chưng cất tri thức	Mạng học trò bắt chước mạng thầy trên ảnh tốt	Sai khác giữa đặc trưng thầy và trò	STFPM
Tự giám sát với lỗi giả	Bộ phân loại học trên lỗi tổng hợp	Đầu ra của bộ phân loại	CutPaste

1.8. Đánh giá mô hình

1.8.1. Các chỉ số phân loại thông thường

Với bài toán phân loại hai lớp (lỗi là lớp dương, tốt là lớp âm), mỗi dự đoán rơi vào một trong bốn ô của ma trận nhầm lẫn: dương thật (TP), dương giả (FP), âm thật (TN), âm giả (FN). Từ đó có các chỉ số thường dùng:

$$\begin{aligned} \text{Accuracy} &= \frac{\text{TP}+\text{TN}}{\text{TP}+\text{TN}+\text{FP}+\text{FN}}, & \text{Precision} &= \frac{\text{TP}}{\text{TP}+\text{FP}}, \\ \text{Recall} &= \frac{\text{TP}}{\text{TP}+\text{FN}}, & F_1 &= \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}. \end{aligned} \tag{1.4}$$

Accuracy là tỷ lệ dự đoán đúng; Precision cho biết trong các mẫu bị báo lỗi có bao nhiêu là lỗi thật; Recall cho biết bắt được bao nhiêu phần lỗi thật; F_1 cân bằng hai chỉ số sau. Với bài toán hồi quy, các chỉ số tương ứng là sai số tuyệt đối trung bình (MAE) và căn sai số bình phương trung bình (RMSE). Tất cả các chỉ số phân loại trên đều cần một ngưỡng quyết định cố định, và Accuracy còn bị lệch khi hai lớp chênh lệch số lượng, như ở mức điểm ảnh, nơi gần như mọi điểm ảnh đều bình thường.

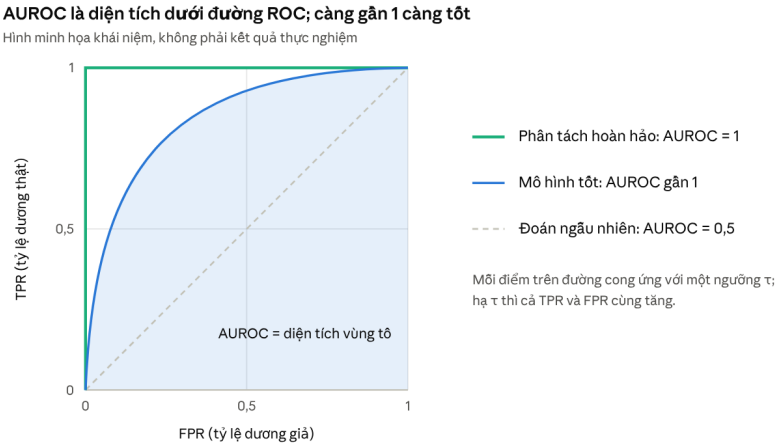
1.8.2. Đường cong ROC và AUROC

Để quyết định tốt hay lỗi, cần một ngưỡng t: ảnh có $s(x) > t$ bị coi là lỗi. Với mỗi ngưỡng, tỷ lệ phát hiện đúng (TPR, chính là Recall) và tỷ lệ báo động giả (FPR) là

$$\text{TPR}(t) = \frac{\text{TP}(t)}{\text{TP}(t)+\text{FN}(t)}, \qquad \text{FPR}(t) = \frac{\text{FP}(t)}{\text{FP}(t)+\text{TN}(t)}. \tag{1.5}$$

Đường cong ROC vẽ TPR theo FPR khi t chạy qua mọi giá trị; AUROC là diện tích dưới đường cong đó [15]. AUROC bằng xác suất một mẫu lỗi chọn ngẫu nhiên có điểm cao hơn một mẫu bình thường chọn ngẫu nhiên. AUROC bằng 1 nghĩa là có ngưỡng tách hoàn toàn hai nhóm;

bằng 0,5 nghĩa là không hơn đoán ngẫu nhiên (Hình 1.7).



Hình 1.7. Đường cong ROC và AUROC (hình minh họa khái niệm, không phải kết quả thực nghiệm)

Đề tài dùng AUROC vì chưa chọn ngưỡng quyết định, cần đánh giá khả năng xếp hạng và so sánh với công bố trên MVTec AD. Khác với accuracy, AUROC không phụ thuộc tỷ lệ mẫu tốt/lỗi khi phân phối điểm của từng lớp giữ nguyên.

Image-AUROC tính trên điểm $s(x)$ và nhãn tốt/lỗi của các ảnh test; pixel-AUROC tính trên giá trị $A_x(u, v)$ và mặt nạ vùng lỗi của mọi điểm ảnh test. Cả hai được tính riêng cho từng loại sản phẩm rồi lấy trung bình, vì điểm của các mô hình khác loại không cùng thang đo. Một hạn chế cần lưu ý: pixel-AUROC được tính trên từng điểm ảnh, nên vùng lỗi lớn đóng góp nhiều hơn vùng lỗi nhỏ.

1.9. Công cụ và thư viện

Bảng 1.2. Công cụ và thư viện sử dụng trong đề tài

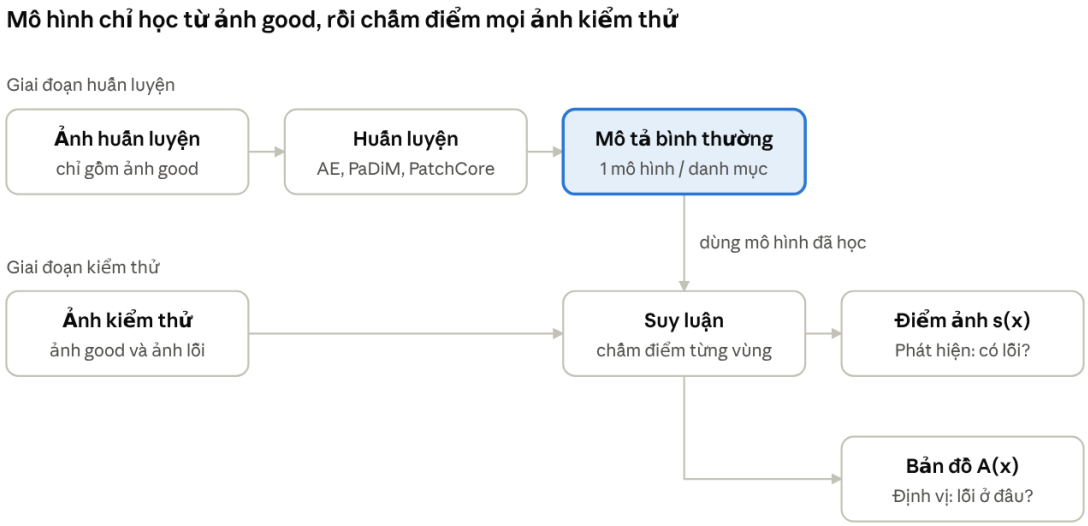
Thư viện	Vai trò trong đề tài
PyTorch, torchvision [7]	Xây dựng và huấn luyện Autoencoder; nạp ResNet-18, Wide-ResNet-50-2 học trước; đọc và biến đổi ảnh
Anomalib [16]	Cài đặt PaDiM và PatchCore (phiên bản 2.7.0)
scikit-learn [17]	Tính image-AUROC và pixel-AUROC
NumPy [18], SciPy	Xử lý mảng bản đồ bất thường, làm mượt Gauss
Matplotlib [19]	Vẽ ảnh, mặt nạ và bản đồ bất thường minh họa
pandas	Ghi và tổng hợp bảng kết quả
Google Colab	Môi trường chạy thực nghiệm với GPU Tesla T4

CHƯƠNG 2. PHƯƠNG PHÁP PHÁT HIỆN BẤT THƯỜNG

Chương này trình bày bài toán cụ thể của đề tài, bộ dữ liệu, giao thức chung, lý do chọn ba phương pháp và cách triển khai từng phương pháp. Mọi cấu hình nêu ở đây là cấu hình đã chạy để ra kết quả ở Chương 3.

2.1. Phát biểu bài toán

Với mỗi loại sản phẩm, mô hình chỉ được học từ ảnh tốt của loại đó, sau đó phải cho mỗi ảnh kiểm thử hai đầu ra: điểm bất thường của ảnh $s(x)$ để trả lời "sản phẩm có lỗi không", và bản đồ bất thường $A_x(u, v)$ để trả lời "lỗi nằm ở đâu" (Hình 2.1; định nghĩa ở Mục 1.7). Mỗi loại có một mô hình riêng, vì "bình thường" của một chai thủy tinh khác hẳn "bình thường" của một tấm thảm. Đề tài đánh giá bằng AUROC, chỉ số không phụ thuộc ngưỡng, nên không chọn ngưỡng quyết định tốt/lỗi.



Hình 2.1. Thiết lập bài toán: học từ ảnh tốt, mỗi ảnh kiểm thử cho điểm ảnh và bản đồ bất thường

2.2. Tổng quan giải pháp

Giải pháp của đề tài là một chuỗi xử lý chung cho cả ba phương pháp; chỉ bước xây dựng mô hình và suy luận là khác nhau. Với mỗi loại sản phẩm:

1. **Đầu vào:** ảnh `train/good` để xây dựng mô hình, ảnh test và mặt nạ để đánh giá (Mục 2.3).
2. **Tiền xử lý:** đổi ảnh về 256×256, chuẩn hóa theo yêu cầu của từng phương pháp, đổi mặt nạ cùng kích thước (Mục 2.4).

3. **Xây dựng mô hình từ ảnh tốt:** huấn luyện Autoencoder; hoặc trích đặc trưng ResNet-18 và ước lượng phân phối Gauss theo vị trí (PaDiM); hoặc trích đặc trưng Wide-ResNet-50-2 và chọn coreset (PatchCore) (Mục 2.6 đến 2.8).
4. **Suy luận:** với mỗi ảnh test, tính sai số tái tạo, khoảng cách Mahalanobis hoặc khoảng cách tới patch gần nhất tại từng vị trí.
5. **Bản đồ và điểm bất thường:** đưa các giá trị cục bộ về bản đồ 256×256, làm mượt, rồi suy ra điểm của ảnh.
6. **Đánh giá:** tính image-AUROC từ điểm ảnh, pixel-AUROC từ bản đồ, ghi kèm thời gian và dung lượng (Mục 2.9).

Mỗi lựa chọn cấu hình trong chuỗi trên đều có lý do (Bảng 2.1). Nguyên tắc chung là giữ giá trị mặc định của bài báo gốc và thư viện khi không có lý do để đổi, và không chỉnh bất kỳ giá trị nào theo kết quả trên tập test.

2.3. Bộ dữ liệu MVTec AD

MVTec AD [1] do công ty MVTec Software công bố năm 2019, là bộ dữ liệu chuẩn cho phát hiện bất thường không giám sát trên ảnh công nghiệp, phát hành theo giấy phép CC BY-NC-SA 4.0 (chỉ dùng cho mục đích phi thương mại). Đường tải chính thức không truy cập được khi thực hiện đề tài, nên nhóm dùng bản sao trên Hugging Face và đã đối chiếu tổng số ảnh, số kiểu lỗi với bài báo gốc.

Bộ dữ liệu gồm 5354 ảnh độ phân giải từ 700×700 đến 1024×1024, thuộc 15 loại: 5 loại bề mặt (texture) và 10 loại vật thể (object); grid, screw, zipper là ảnh xám. Mỗi loại có ba thư mục: `train/good` chỉ chứa ảnh tốt, `test/<kiểu lỗi>` chứa ảnh tốt và ảnh lỗi để kiểm thử, `ground_truth/<kiểu lỗi>` chứa mặt nạ nhị phân đánh dấu vùng lỗi của từng ảnh lỗi (Bảng 2.2).

Bốn đặc điểm của bộ dữ liệu được dùng để phân tích kết quả ở Chương 3:

- Với texture, mẫu bình thường lặp lại khắp ảnh và lỗi có thể nằm ở bất kỳ đâu.
- Phần lớn vật thể được chụp ở tư thế gần cố định (bottle, cable, transistor), nhưng screw, hazelnut và metal_nut xoay ngẫu nhiên giữa các ảnh, kể cả ảnh tốt.
- Một số kiểu lỗi mang tính logic (Mục 1.7): transistor/misplaced (thiếu linh kiện), cable/cable_swap (đổi chỗ dây), metal_nut/flip (lật mặt).
- Toothbrush chỉ có 60 ảnh huấn luyện, ít nhất trong bộ dữ liệu.

2.4. Tiền xử lý và giao thức chung

Để so sánh công bằng, ba phương pháp dùng chung cách xử lý dữ liệu và cách đánh giá; chỉ phần mô hình bên trong là khác nhau.

1. **Kích thước ảnh.** Ảnh được đổi về 256×256 bằng nội suy song tuyến, không cắt ảnh. Bài báo PaDiM và PatchCore cắt vùng giữa 224×224; nhóm không cắt để không mất

Bảng 2.1. Các lựa chọn cấu hình và lý do

Lựa chọn	Giá trị	Lý do
Kích thước ảnh	256×256, không cắt	Đủ chi tiết để thấy lỗi nhỏ mà chi phí tính toán vừa phải; là kích thước bài báo PaDiM và PatchCore đổi ảnh về trước khi cắt. Không cắt vùng giữa để giữ lỗi sát mép và để bản đồ khớp toàn bộ mặt nạ. Chia hết cho 2^5 nên Autoencoder thu gọn đúng về 8×8.
Số mô hình	Một mô hình cho mỗi loại	"Bình thường" của mỗi loại sản phẩm khác nhau; đây cũng là giao thức chuẩn của MVTec AD.
Autoencoder	Tự xây dựng, học từ đầu	Đại diện cho hướng học từ ảnh mà không dùng đặc trưng học trước, làm mốc để thấy giá trị của đặc trưng học trước.
Backbone PaDiM	ResNet-18, `layer1`–`layer3`, $d = 100$	Với Wide-ResNet-50-2 và $d = 550$, mô hình PaDiM cần khoảng 4,6 GB cho mỗi loại; ResNet-18 chỉ cần khoảng 158 MB. Ba tầng kết hợp chi tiết cục bộ với ngữ cảnh.
Backbone PatchCore	Wide-ResNet-50-2, `layer2`–`layer3`	Cấu hình mặc định của bài báo và Anomalib. Dung lượng PatchCore chỉ phụ thuộc kích thước coreset nên backbone lớn vẫn vừa GPU 16 GB. Tầng giữa cân bằng chi tiết và ngữ nghĩa; `layer4` quá thô để định vị.
Coreset	10% số vector	Mặc định của bài báo; giữ được gần như toàn bộ độ chính xác trong khi giảm 10 lần bộ nhớ và thời gian tìm láng giềng.
Điểm ảnh của AE, PaDiM	Giá trị lớn nhất của bản đồ	Một sản phẩm chỉ cần một vết lỗi nhỏ là đã lỗi, nên điểm của ảnh phải nhạy với vùng sai lệch mạnh nhất chứ không phải trung bình cả ảnh.
Điểm ảnh của PatchCore	Hiệu chỉnh trọng số với $b = 9$	Cách tính của bài báo gốc: giảm điểm khi patch khác thường thực ra gần nhiều mẫu bình thường hiếm gặp, nhờ đó giảm báo động giả.
Làm mượt bản đồ	Bộ lọc Gauss $\sigma = 4$	Giảm nhiễu từng điểm ảnh để giá trị lớn nhất không bị một điểm lẻ quyết định; là giá trị mặc định của Anomalib, dùng chung cho cả ba phương pháp.
Tăng cường dữ liệu	Không dùng	Xoay, lật có thể biến ảnh tốt thành ảnh giống lỗi (metal_nut lật mặt là một kiểu lỗi).

Bảng 2.2. Số ảnh và các kiểu lỗi của MVTec AD theo loại

Nhóm	Loại	Train	Test tốt	Test lỗi	Kiểu lỗi
Texture	carpet	280	28	89	color, cut, hole, metal_contamination, thread
	grid	264	21	57	bent, broken, glue, metal_contamination, thread
	leather	245	32	92	color, cut, fold, glue, poke
	tile	230	33	84	crack, glue_strip, gray_stroke, oil, rough
	wood	247	19	60	color, combined, hole, liquid, scratch
Object	bottle	209	20	63	broken_large, broken_small, contamination
	cable	224	58	92	bent_wire, cable_swap, combined, cut_inner_insulation, cut_outer_insulation, missing_cable, missing_wire, poke_insulation
	capsule	219	23	109	crack, faulty_imprint, poke, scratch, squeeze
	hazelnut	391	40	70	crack, cut, hole, print
	metal_nut	220	22	93	bent, color, flip, scratch
	pill	267	26	141	color, combined, contamination, crack, faulty_imprint, pill_type, scratch
	screw	320	41	119	manipulated_front, scratch_head, scratch_neck, thread_side, thread_top
	toothbrush	60	12	30	defective
	transistor	213	60	40	bent_lead, cut_lead, damaged_case, misplaced
	zipper	240	32	119	broken_teeth, combined, fabric_border, fabric_interior, rough, split_teeth, squeezed_teeth
Tổng		3629	467	1258	73

lỗi nằm sát mép ảnh và để bản đồ bất thường khớp toàn bộ mặt nạ.

2. **Ảnh xám.** Ảnh của grid, screw, zipper được nhân thành 3 kênh để dùng chung mạng với ảnh màu.
3. **Chuẩn hóa giá trị điểm ảnh.** PaDiM và PatchCore chuẩn hóa theo trung bình (0,485; 0,456; 0,406) và độ lệch chuẩn (0,229; 0,224; 0,225) của ImageNet, đúng phân phối mà backbone đã học. Autoencoder giữ giá trị trong $[0, 1]$ để so sánh trực tiếp với đầu ra sigmoid.
4. **Mặt nạ.** Mặt nạ được đổi kích thước bằng nội suy lân cận gần nhất rồi nhị phân hóa lại (ngưỡng 0,5); ảnh test tốt được gán mặt nạ toàn 0.
5. **Không tăng cường dữ liệu.** Cả ba phương pháp đều không xoay hay lật ảnh. Với một số loại, phép biến đổi này tạo ra ảnh trông giống lỗi (metal_nut lật mặt là một kiểu lỗi) và đưa mẫu "bình thường" không có thật vào mô hình.
6. **Chia dữ liệu.** Giữ nguyên cách chia train/test của bộ dữ liệu. Mô hình chỉ học từ `train/good`; ảnh test chỉ dùng để đánh giá lần cuối. Siêu tham số giữ theo giá trị mặc định của bài báo gốc và thư viện, không chỉnh theo kết quả trên tập test.
7. **Đánh giá.** Image-AUROC và pixel-AUROC được tính bằng cùng một module (hàm `roc_auc_score` của scikit-learn [20]) riêng cho từng loại, rồi lấy trung bình theo loại. Seed ngẫu nhiên cố định là 42.

2.5. Lựa chọn phương pháp

Ba phương pháp được chọn vì ba lý do. Thứ nhất, chúng thuộc ba hướng khác nhau trong Bảng 1.1: tái tạo, mô hình hóa phân phối và bộ nhớ đặc trưng, nên so sánh chúng cho thấy ảnh hưởng của không gian so sánh (điểm ảnh hay đặc trưng) và cách mô tả "bình thường". Thứ hai, chúng tạo thành một chuỗi mà mỗi phương pháp khắc phục hạn chế chính của phương pháp trước (Hình 2.2): PaDiM chuyển từ so sánh điểm ảnh sang so sánh đặc trưng học trước; PatchCore bỏ ràng buộc theo vị trí và giả định phân phối Gauss của PaDiM. Thứ ba, cả ba đều làm được trong phạm vi môn học: Autoencoder đủ đơn giản để tự xây dựng, PaDiM và PatchCore có sẵn trong thư viện Anomalib [16], và cả ba đều có kết quả công bố trên MVTec AD để đối chiếu.

Từ cơ chế của ba phương pháp, nhóm đặt ra ba giả thuyết và kiểm chứng ở Chương 3:

(G1) PaDiM và PatchCore vượt Autoencoder ở hầu hết các loại, vì đặc trưng học trước phân biệt lỗi tốt hơn sai số từng điểm ảnh.

(G2) Khoảng cách giữa PatchCore và PaDiM lớn nhất ở các loại vật thể xoay (screw, hazelnut, metal_nut), vì PaDiM gắn thống kê với vị trí còn PatchCore thì không.

(G3) Autoencoder làm tốt hơn ở các loại có nền đơn giản và lỗi lớn, kém ở các loại có lỗi nhỏ hoặc nhiều chi tiết khó tái tạo.



Hình 2.2. Ba phương pháp theo không gian biểu diễn, cách mô tả "bình thường" và cách tính điểm

2.6. Autoencoder

2.6.1. Nguyên lý

Autoencoder gồm bộ mã hóa f nén ảnh x thành vector tiềm ẩn z và bộ giải mã g dựng lại ảnh từ z [5]:

$$z = f(x), \qquad \hat{x} = g(z) = g\big(f(x)\big). \tag{2.1}$$

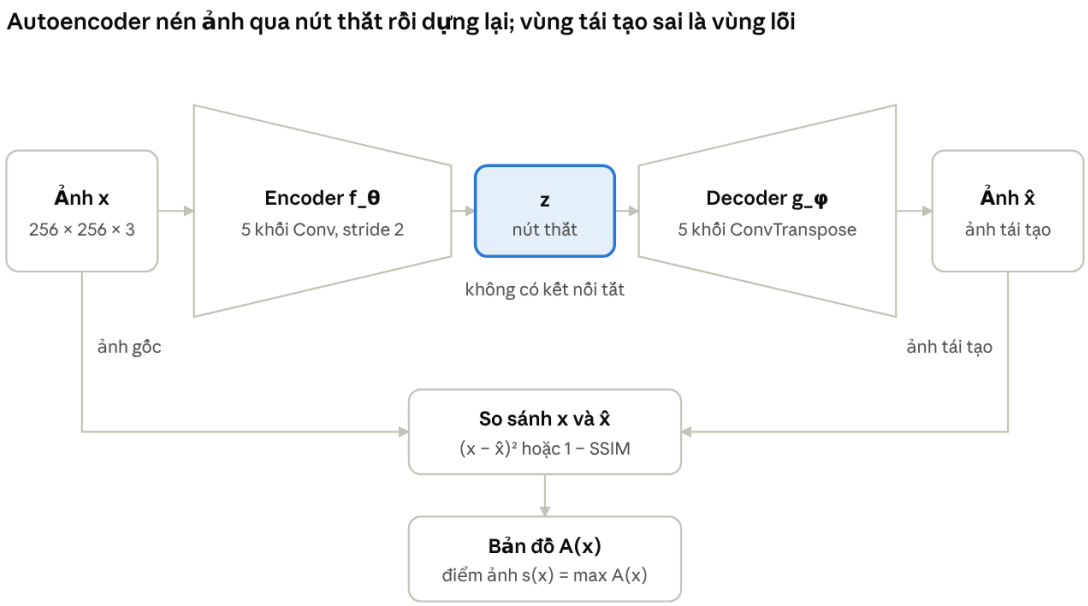
Vector z nhỏ hơn nhiều so với ảnh, nên mạng không chép được nguyên ảnh mà phải học những cấu trúc chung của ảnh tốt. Khi gặp ảnh lỗi, mạng dựng lại theo kiểu ảnh tốt; vùng lỗi bị dựng sai nên có sai số tái tạo lớn (Hình 2.3).

2.6.2. Kiến trúc

Nhóm tự xây dựng một Autoencoder tích chập bằng PyTorch (Bảng 2.3). Bộ mã hóa gồm 5 lớp tích chập bước 2 thu ảnh từ 256×256 về 8×8, rồi một lớp kết nối đầy đủ nén $256 \times 8 \times 8 = 16384$ giá trị thành vector 256 chiều. Bộ giải mã đối xứng với bộ mã hóa. Mô hình có khoảng 11,9 triệu tham số (45 MB); hai lớp kết nối đầy đủ ở cổ chai chiếm khoảng 70%.

Hai lựa chọn thiết kế quyết định khả năng phát hiện lỗi:

- **Không có kết nối tắt giữa bộ mã hóa và bộ giải mã** như kiến trúc U-Net. Kết nối tắt cho phép chép thẳng chi tiết từ đầu vào sang đầu ra, khi đó cả vùng lỗi cũng được dựng lại đúng và tín hiệu bất thường mất đi.
- **Cổ chai là một vector**, không phải bản đồ không gian. Cổ chai quá rộng thì mạng dựng lại được cả lỗi; quá hẹp thì ảnh tốt cũng bị dựng mờ và gây báo động giả. Vector 256



Hình 2.3. Autoencoder và cách tính bản đồ bất thường (đề tài dùng sai số bình phương, không dùng SSIM)

Bảng 2.3. Kiến trúc Autoencoder (mỗi lớp tích chập dùng bộ lọc 4×4, kèm chuẩn hóa theo lô và LeakyReLU, trừ lớp ra)

Phần	Lớp	Kích thước đầu ra
Mã hóa	5 lớp tích chập bước 2, số kênh 32, 64, 128, 256, 256	$256 \times 8 \times 8$
	Kết nối đầy đủ $16384 \to 256$	256
Giải mã	Kết nối đầy đủ $256 \to 16384$	$256 \times 8 \times 8$
	4 lớp tích chập chuyển vị bước 2, số kênh 256, 128, 64, 32	$32 \times 128 \times 128$
	Tích chập chuyển vị bước 2 về 3 kênh, sigmoid	$3 \times 256 \times 256$

chiều là điểm cân bằng nhóm chọn và giữ cố định cho cả 15 loại.

2.6.3. Huấn luyện

Hàm mất mát là sai số bình phương trung bình trên mọi điểm ảnh và kênh màu:

$$L(x, \hat{x}) = \frac{1}{HWC} \sum_{u=1}^{H} \sum_{v=1}^{W} \sum_{c=1}^{C} \big[x(u, v, c) - \hat{x}(u, v, c)\big]^2. \tag{2.2}$$

Mỗi loại được huấn luyện 100 epoch bằng Adam, tốc độ học 10^{-3}, weight decay 10^{-5}, lô 16 ảnh. Hàm mất mát dựa trên SSIM [21], so sánh cấu trúc cục bộ thay vì từng điểm ảnh, là một lựa chọn khác được báo cáo tốt hơn ở một số loại; đề tài chưa thử và ghi nhận ở phần Hạn chế.

2.6.4. Bản đồ và điểm bất thường

Bản đồ bất thường là sai số bình phương tại từng điểm ảnh, lấy trung bình theo kênh:

$$A(u, v) = \frac{1}{C} \sum_{c=1}^{C} \big[x(u, v, c) - \hat{x}(u, v, c)\big]^2. \tag{2.3}$$

Bản đồ được làm mượt bằng bộ lọc Gauss với $\sigma = 4$ để giảm nhiễu từng điểm ảnh, rồi lấy giá trị lớn nhất làm điểm của ảnh.

2.6.5. Ưu điểm và hạn chế

Autoencoder đơn giản, dễ giải thích (xem trực tiếp được ảnh dựng lại), mô hình nhỏ và không phụ thuộc mạng học trước. Hạn chế nằm ở chính không gian so sánh: ảnh dựng lại thường mờ ở cạnh và kết cấu chi tiết, nên sai số xuất hiện cả ở vùng bình thường; mạng đủ mạnh có thể dựng lại được cả lỗi; và sai khác từng điểm ảnh nhạy với ánh sáng hay dịch chuyển nhỏ nhưng kém nhạy với lỗi mang tính ngữ nghĩa. Ngoài ra, mạng phải huấn luyện từ đầu và kết quả phụ thuộc nhiều vào siêu tham số.

2.7. PaDiM

2.7.1. Nguyên lý

PaDiM [2] chia ảnh thành lưới các vùng nhỏ (patch). Mỗi patch được mô tả bằng vector đặc trưng lấy từ mạng tích chập học trước. Phương pháp giả định rằng, trên ảnh tốt, các vector tại cùng một vị trí tuân theo một phân phối Gauss nhiều chiều; patch nào lệch xa phân phối của vị trí đó là bất thường (Hình 2.4). PaDiM không lan truyền ngược: phần "huấn luyện" chỉ là một lượt duyệt ảnh tốt để ước lượng thống kê.



Hình 2.4. Quy trình PaDiM (với ảnh 256×256 của đề tài, lưới là 64×64 và $d = 100$)

2.7.2. Trích xuất đặc trưng

Đặc trưng lấy từ ba tầng `layer1`, `layer2`, `layer3` của ResNet-18 (Mục 1.3). Tầng nông giữ chi tiết cục bộ, tầng sâu hơn mang ngữ cảnh rộng; kết hợp cả ba để bắt được cả lỗi nhỏ lẫn lỗi lớn. Bản đồ của `layer2` và `layer3` được phóng về kích thước của `layer1` rồi ghép theo kênh, cho lưới 64×64 vị trí, mỗi vị trí một vector $64 + 128 + 256 = 448$ chiều.

Kích thước ma trận hiệp phương sai tăng theo bình phương số chiều, nên PaDiM giữ lại $d = 100$ chiều chọn ngẫu nhiên (cố định theo seed, dùng chung cho huấn luyện và kiểm thử). Bài báo gốc cho thấy chọn ngẫu nhiên cho kết quả tương đương phân tích thành phần chính nhưng nhanh hơn nhiều [2].

2.7.3. Ước lượng phân phối và tính điểm

Gọi $z_k(p)$ là vector đặc trưng của ảnh tốt thứ k tại vị trí p và N là số ảnh tốt:

$$\mu_p = \frac{1}{N} \sum_{k=1}^{N} z_k(p), \tag{2.4}$$

$$\Sigma_p = \frac{1}{N-1} \sum_{k=1}^{N} \left[z_k(p) - \mu_p\right] \left[z_k(p) - \mu_p\right]^\top + \varepsilon I. \tag{2.5}$$

Thành phần εI với $\varepsilon = 0{,}01$ bảo đảm Σ_p khả nghịch; điều này cần thiết vì mỗi loại chỉ có 60 đến 391 ảnh tốt, có loại ít hơn số chiều d. Với ảnh kiểm thử, điểm tại p là khoảng cách Mahalanobis từ $z(p)$ tới phân phối bình thường:

$$d_p = \sqrt{\left[z(p) - \mu_p\right]^\top \Sigma_p^{-1} \left[z(p) - \mu_p\right]}. \tag{2.6}$$

Khác khoảng cách Euclid, khoảng cách này tính đến độ biến thiên và tương quan giữa các chiều: lệch trên một chiều vốn ổn định bị phạt nặng hơn lệch cùng độ lớn trên một chiều vốn dao động nhiều. Bản đồ 64×64 các giá trị d_p được phóng về 256×256 và làm mượt Gauss $\sigma = 4$; điểm ảnh là giá trị lớn nhất của bản đồ.

2.7.4. Triển khai và bộ nhớ

Nhóm dùng lớp PaDiM của Anomalib 2.7.0 [16], giữ nguyên trọng số ResNet-18 học trước. Mô hình của mỗi loại gồm 4096 vector trung bình và 4096 ma trận nghịch đảo 100×100, khoảng 158 MB ở dạng số thực 32 bit. Đây cũng là lý do dùng ResNet-18 thay vì Wide-ResNet-50-2 như bài báo gốc: với Wide-ResNet-50-2 và $d = 550$, cùng lưới 64×64 cần 4096×550^2 phần tử, khoảng 4,6 GB cho mỗi loại.

2.7.5. Ưu điểm và hạn chế

PaDiM không cần huấn luyện mạng, chỉ một lượt duyệt dữ liệu nên xây dựng mô hình nhanh, và tận dụng được đặc trưng học trước nên khoanh vùng lỗi tốt. Hạn chế chính là giả định ảnh đã căn chỉnh: khi vật thể xoay hay dịch chuyển, phân phối tại một vị trí trộn nhiều bộ phận khác nhau và kém phân biệt hơn. Ngoài ra, bộ nhớ tăng theo bình phương số chiều, và một phân phối Gauss duy nhất không mô tả đủ khi tại cùng vị trí có nhiều kiểu bình thường khác nhau.

2.8. PatchCore

2.8.1. Nguyên lý

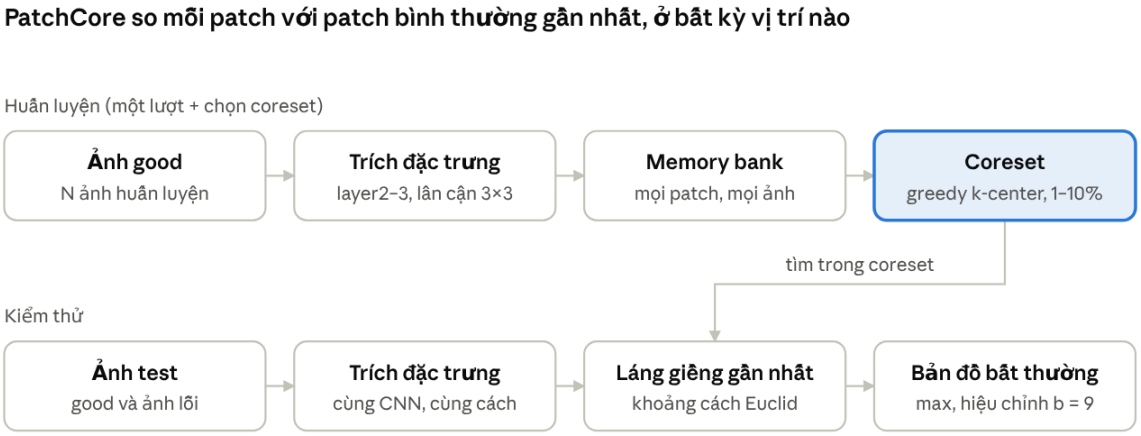
PatchCore [3] coi "bình thường" là những gì đã xuất hiện trong ảnh tốt: một patch kiểm thử là bình thường nếu nó giống ít nhất một patch của ảnh tốt, ở bất kỳ vị trí nào trong bất kỳ ảnh nào (Hình 2.5). So với PaDiM, phương pháp có hai khác biệt: không phụ thuộc vị trí, và không giả định dạng phân phối mà dùng trực tiếp các vector mẫu.

2.8.2. Đặc trưng patch có ngữ cảnh lân cận

Đặc trưng lấy từ `layer2` (512 kênh, lưới 32×32) và `layer3` (1024 kênh, 16×16) của Wide-ResNet-50-2. `layer1` quá gần mức điểm ảnh, `layer4` quá thiên về phân loại ImageNet. Tại mỗi vị trí, đặc trưng được lấy trung bình trên lân cận 3×3 để mỗi vector mang thông tin vùng xung quanh và bền hơn với dịch chuyển nhỏ. Bản đồ của `layer3` được phóng về 32×32 rồi ghép với `layer2`, nên mỗi ảnh cho 1024 vector 1536 chiều.

2.8.3. Bộ nhớ và chọn coreset

Gom vector của mọi patch trong mọi ảnh tốt được bộ nhớ đầy đủ $\mathcal{M}$: một loại vài trăm ảnh có vài trăm nghìn vector, tìm láng giềng gần nhất trên đó rất chậm. PatchCore giữ lại một tập con



Hình 2.5. Quy trình PatchCore (đề tài giữ coreset 10%)

$\mathcal{M}_C$ sao cho mọi vector trong $\mathcal{M}$ đều có một đại diện gần nó trong $\mathcal{M}_C$:

$$\mathcal{M}_C^* = \arg \min_{\mathcal{M}_C \subset \mathcal{M}} \max_{m \in \mathcal{M}} \min_{n \in \mathcal{M}_C} \|m - n\|_2. \tag{2.7}$$

Tìm nghiệm đúng của bài toán này quá tốn kém, nên PatchCore giải gần đúng bằng thuật toán tham lam k-center [22]:

1. Chiếu ngẫu nhiên mọi vector trong $\mathcal{M}$ xuống ít chiều hơn, để tính khoảng cách nhanh hơn mà vẫn giữ gần đúng khoảng cách giữa chúng.
2. Khởi tạo $\mathcal{M}_C$ bằng một vector chọn ngẫu nhiên.
3. Với mỗi vector trong $\mathcal{M}$, cập nhật khoảng cách tới vector gần nhất của $\mathcal{M}_C$.
4. Thêm vào $\mathcal{M}_C$ vector có khoảng cách đó lớn nhất, tức vector đang được đại diện kém nhất.
5. Lặp bước 3 và 4 đến khi $\mathcal{M}_C$ đủ kích thước mong muốn.

Khác lấy mẫu ngẫu nhiên, cách chọn này giữ được cả những kiểu patch bình thường hiếm gặp, nhờ đó giảm báo động giả. Bài báo gốc cho thấy giữ 1% đến 10% số vector vẫn cho kết quả gần như bộ nhớ đầy đủ [3].

Đề tài giữ 10% số vector. Ví dụ hazelnut có 391 ảnh tốt, tức khoảng 400 nghìn vector; coreset còn khoảng 40 nghìn vector, chiếm khoảng 235 MB. Vì mỗi bước chọn phụ thuộc bước trước, thời gian chọn coreset tăng nhanh theo số ảnh.

2.8.4. Tính điểm

Điểm của một patch q trong ảnh kiểm thử là khoảng cách tới patch bình thường gần nhất trong coreset:

$$a(q) = \min_{m \in \mathcal{M}_C} \|q - m\|_2. \tag{2.8}$$

Bản đồ 32×32 các giá trị $a(q)$ được phóng về 256×256 và làm mượt Gauss $\sigma = 4$. Điểm của ảnh không chỉ là patch có $a(q)$ lớn nhất. Gọi q^* là patch đó, m^* là láng giềng gần nhất của nó và $\mathcal{N}_b(m^*)$ là b láng giềng gần nhất của m^* trong coreset; điểm được nhân thêm một trọng số:

$$s = \left(1 - \frac{\exp\|q^* - m^*\|_2}{\sum_{m \in \mathcal{N}_b(m^*)} \exp\|q^* - m\|_2}\right) a(q^*). \tag{2.9}$$

Trọng số này giảm điểm khi m^* nằm trong vùng dày đặc của bộ nhớ, tức q^* gần nhiều mẫu bình thường chứ không chỉ một mẫu. Đề tài dùng $b = 9$, đúng như cài đặt của Anomalib 2.7.0.

2.8.5. Ưu điểm và hạn chế

PatchCore không phụ thuộc vị trí nên chịu được vật thể xoay hay lệch, không cần huấn luyện mạng và không giả định dạng phân phối. Đổi lại, mỗi patch kiểm thử phải tìm láng giềng gần nhất trong coreset nên suy luận chậm hơn PaDiM, và bước chọn coreset tốn thời gian khi nhiều ảnh. Việc bỏ thông tin vị trí cũng có mặt trái: từng patch của một vật thể thiếu linh kiện hay lắp sai chỗ vẫn có thể giống một patch bình thường nào đó, nên PatchCore dễ bỏ sót lỗi logic. Cũng như PaDiM, chất lượng phụ thuộc vào đặc trưng học từ ImageNet.

2.9. Tiêu chí so sánh và quy trình thực nghiệm

Độ chính xác chưa đủ để chọn phương pháp cho ứng dụng thực tế, nên ba phương pháp được so sánh trên bốn nhóm tiêu chí (Bảng 2.4).

Bảng 2.4. Tiêu chí so sánh và cách đo

Tiêu chí	Đại lượng đo	Trả lời câu hỏi
Độ chính xác	Image-AUROC, pixel-AUROC theo từng loại và trung bình	Phát hiện và khoanh vùng lỗi tốt đến đâu
Thời gian	Thời gian xây dựng mô hình mỗi loại; thời gian suy luận mỗi ảnh	Có theo kịp tốc độ dây chuyền không
Dung lượng	Kích thước mô hình lưu lại: trọng số (AE), trung bình và ma trận nghịch đảo (PaDiM), coreset (PatchCore)	Cần bao nhiêu bộ nhớ khi triển khai
Định tính	Bản đồ bất thường trên ảnh từng kiểu lỗi, cùng thang màu cho mỗi loại	Vì sao phương pháp đúng hoặc sai

Bảng 2.5 tóm tắt cấu hình của ba phương pháp; Mục 2.10 trình bày cách cài đặt và chạy.

Bảng 2.5. Cấu hình đã chạy của ba phương pháp

Phương pháp	Cấu hình
Autoencoder	Ảnh 256×256 trong $[0, 1]$; cổ chai vector 256 chiều; MSE; Adam, tốc độ học 10^{-3}, weight decay 10^{-5}, lô 16, 100 epoch; làm mượt $\sigma = 4$; điểm ảnh là giá trị lớn nhất
PaDiM	ResNet-18 học trước; `layer1`–`layer3`, giữ ngẫu nhiên $d = 100$ chiều; $\varepsilon = 0{,}01$; làm mượt $\sigma = 4$; điểm ảnh là giá trị lớn nhất
PatchCore	Wide-ResNet-50-2 học trước; `layer2`–`layer3`, lân cận 3×3; coreset 10%; $b = 9$; làm mượt $\sigma = 4$

2.10. Cài đặt và chạy thực nghiệm

2.10.1. Tổ chức mã nguồn

Mã nguồn viết bằng Python, tổ chức theo nguyên tắc "cố định cách đánh giá, tự do cách triển khai". Hai phần dùng chung cho cả ba phương pháp là module đọc dữ liệu (tiền xử lý như Mục 2.4) và module đánh giá (tính AUROC, vẽ hình, ghi kết quả). Mỗi phương pháp là một module riêng, chỉ cần nhận ảnh từ module dữ liệu và trả về bản đồ cùng điểm bất thường đúng kích thước 256×256. Nhờ vậy, đổi cách cài đặt một phương pháp không làm thay đổi cách đánh giá, và kết quả của ba phương pháp so sánh được với nhau. Autoencoder do nhóm tự viết bằng PyTorch; PaDiM và PatchCore dùng trực tiếp lớp mô hình của Anomalib 2.7.0, nhưng nhận ảnh từ module dữ liệu của nhóm thay vì bộ nạp dữ liệu của thư viện, để bảo đảm cùng tiền xử lý với Autoencoder. Cấu trúc thư mục chi tiết nằm ở Phụ lục.

2.10.2. Quy trình chạy một loại sản phẩm

Với mỗi phương pháp và mỗi loại, chương trình thực hiện:

1. Đặt lại seed ngẫu nhiên bằng 42, để kết quả của mỗi loại không phụ thuộc thứ tự chạy.
2. Đọc ảnh `train/good` và toàn bộ tập test của loại đó qua module dữ liệu.
3. Xây dựng mô hình từ ảnh tốt: huấn luyện mạng (Autoencoder), ước lượng phân phối (PaDiM) hoặc trích đặc trưng và chọn coreset (PatchCore).
4. Suy luận trên toàn bộ tập test để lấy bản đồ bất thường 256×256 và điểm của từng ảnh.
5. Tính image-AUROC và pixel-AUROC bằng module đánh giá chung.
6. Ghi một dòng vào tệp kết quả chung (loại, phương pháp, hai chỉ số AUROC, thời gian xây dựng, thời gian suy luận, dung lượng mô hình, cấu hình) và lưu ảnh minh họa cho từng kiểu lỗi. Với Autoencoder, trọng số mạng của từng loại cũng được lưu lại.

2.10.3. Cách chạy trên Colab

Thực nghiệm đầy đủ chạy trên Google Colab; cấu hình phần cứng, phiên bản thư viện và các thiết lập khác được tổng hợp ở Mục 3.1. Dữ liệu và kết quả đặt trên Google Drive, nên khi phiên

Colab bị ngắt thì không mất gì. Mỗi loại chạy xong được ghi ngay vào tệp kết quả; khi chạy lại, chương trình bỏ qua các loại đã có kết quả với cùng cấu hình và chạy tiếp từ loại còn thiếu. Trước khi chạy đủ 15 loại, mỗi phương pháp được chạy thử trên loại bottle ở máy cá nhân (GPU RTX 3050, 4 GB) để kiểm tra toàn bộ quy trình.

2.10.4. Các vấn đề kỹ thuật đã xử lý

- **PaDiM lỗi trên GPU Colab.** Phép nghịch đảo đồng thời 4096 ma trận hiệp phương sai trên GPU Tesla T4 gây lỗi truy cập bộ nhớ trong thư viện CUDA đi kèm PyTorch trên Colab. Nhóm tính trung bình, hiệp phương sai và nghịch đảo trên CPU rồi mới chuyển kết quả lên GPU để suy luận. Kết quả không đổi: loại bottle cho cùng image-AUROC 0,9976 và pixel-AUROC 0,9776 trước và sau khi sửa.
- **PatchCore trên GPU 4 GB.** Khi chạy thử ở máy cá nhân, bộ nhớ đặc trưng đầy đủ không vừa GPU. Nhóm giữ toàn bộ đặc trưng trên RAM và chỉ đưa bản chiếu ngẫu nhiên lên GPU khi chọn coreset; thuật toán không đổi.
- **Autoencoder đọc lại ảnh mỗi epoch.** Cách nạp dữ liệu thông thường giải nén lại từng ảnh PNG độ phân giải cao từ Google Drive ở mỗi epoch, tức 100 lần cho mỗi ảnh. Nhóm đọc và thu nhỏ ảnh huấn luyện một lần, giữ sẵn trên GPU (tối đa khoảng 300 MB với 391 ảnh), rồi xáo và chia lô ngay trên GPU. Cấu hình huấn luyện giữ nguyên; 100 epoch trên bottle còn khoảng 85 giây trên RTX 3050.

2.10.5. Đo thời gian và vẽ hình minh họa

Với Autoencoder, thời gian xây dựng chỉ tính vòng huấn luyện, sau khi đã đọc và chuyển toàn bộ ảnh huấn luyện lên thiết bị. Với PaDiM và PatchCore, thời gian này tính từ lúc duyệt ảnh huấn luyện đến khi hoàn tất trích đặc trưng, ước lượng phân phối hoặc chọn coreset. Thời gian suy luận là tổng thời gian chạy mô hình trên tập test chia cho số ảnh test. Trước và sau mỗi lần đo, chương trình chờ GPU làm xong mọi phép tính để số đo không bị sai do GPU chạy bất đồng bộ. Với PaDiM và PatchCore, thời gian suy luận chỉ gồm lượt chạy mô hình; với Autoencoder, số đo còn gồm cả thời gian đọc ảnh test. Khác biệt này được lưu ý khi so sánh ở Chương 3.

Hình minh họa gồm ảnh gốc, mặt nạ lỗi và bản đồ bất thường của ảnh đầu tiên trong mỗi kiểu lỗi. Thang màu của bản đồ cố định cho mỗi phương pháp và mỗi loại, lấy từ phân vị 1% và 99,5% của mọi bản đồ test, nên các ảnh cùng loại so sánh được với nhau mà không bị chuẩn hóa riêng từng ảnh.

CHƯƠNG 3. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

3.1. Thiết lập thực nghiệm

Bảng 3.1 và Bảng 3.2 tổng hợp mọi thông tin cần để chạy lại thực nghiệm. Kết quả trong chương này lấy từ lần chạy đầy đủ trên Google Colab ngày 25/10/2026; máy cá nhân chỉ dùng để chạy thử trên loại bottle trước đó và để kiểm tra độ lặp lại (Mục 3.7).

Bảng 3.1. Môi trường chạy thực nghiệm

Hạng mục	Chạy chính thức (Google Colab)	Chạy thử (máy cá nhân)
Phần cứng	GPU NVIDIA Tesla T4 16 GB; RAM 12,7 GB	GPU NVIDIA RTX 3050 Laptop 4 GB; RAM 16 GB
Python	3.13	3.11.9
PyTorch	Bản cài sẵn của Colab (dựng cho CUDA 13); phiên bản cụ thể không được ghi lại	2.11.0 (CUDA 12.8), torchvision 0.26.0
Anomalib	2.7.0 (cố định trong `requirements.txt`)	2.7.0
Thư viện khác	scikit-learn, SciPy, NumPy bản mới nhất khi cài	scikit-learn 1.9.1, SciPy 1.17.1, NumPy 2.4.6

Bảng 3.2. Các thiết lập dùng chung cho mọi lần chạy

Hạng mục	Thiết lập
Dữ liệu	MVTec AD, đủ 15 loại, chia train/test gốc (Bảng 2.2); ảnh và mặt nạ đổi về 256×256 (Mục 2.4)
Cấu hình	Bảng 2.5; lý do từng lựa chọn ở Bảng 2.1
Seed	42, đặt lại trước mỗi loại; mỗi phương pháp chạy một lần
Đo thời gian	Xây dựng mô hình: Autoencoder chỉ tính vòng huấn luyện, không gồm bước nạp ảnh ban đầu; PaDiM và PatchCore tính cả đọc ảnh, trích đặc trưng và fit mô hình. Suy luận: trung bình trên mỗi ảnh test; với PaDiM và PatchCore chỉ tính lượt chạy mô hình, với Autoencoder có tính cả đọc ảnh (Mục 2.10)
Mã nguồn	Commit `a839dd0` (PaDiM, PatchCore) và `37fb3e5` (Autoencoder); lệnh chạy ở Phụ lục

3.2. Kết quả tổng hợp

Bảng 3.3 cho kết quả trung bình theo nhóm loại.

Thứ tự PatchCore, PaDiM, Autoencoder giữ nguyên ở cả hai chỉ số và cả hai nhóm loại. Khoảng cách lớn nhất là giữa Autoencoder và hai phương pháp dùng đặc trưng học trước: 0,756 so với 0,890 và 0,983 ở image-AUROC. Với cấu hình đã chạy, một mạng tái tạo tự học trên vài trăm ảnh tốt phát hiện lỗi kém hơn rõ rệt so với việc dùng lại đặc trưng ImageNet.

Bảng 3.3. AUROC trung bình theo nhóm loại (cao nhất in đậm)

Chỉ số	Nhóm	Autoencoder	PaDiM	PatchCore
Image-AUROC	Texture (5 loại)	0,784	0,945	**0,993**
	Object (10 loại)	0,741	0,863	**0,977**
	Cả 15 loại	0,756	0,890	**0,983**
Pixel-AUROC	Texture	0,781	0,944	**0,970**
	Object	0,875	0,968	**0,985**
	Cả 15 loại	0,843	0,960	**0,980**

3.3. Kết quả theo từng loại

Bảng 3.4. AUROC theo từng loại (AE: Autoencoder; cao nhất mỗi chỉ số in đậm)

	Image-AUROC			Pixel-AUROC		
Loại	AE	PaDiM	PatchCore	AE	PaDiM	PatchCore
carpet	0,553	**0,988**	0,988	0,805	0,989	**0,991**
grid	0,857	0,829	**0,990**	0,844	0,916	**0,981**
leather	0,807	0,999	**1,000**	0,916	0,987	**0,992**
tile	0,766	0,943	**1,000**	0,559	0,906	**0,955**
wood	0,938	0,966	**0,987**	0,780	0,922	**0,930**
bottle	0,926	0,998	**1,000**	0,935	0,978	**0,986**
cable	0,579	0,838	**0,988**	0,877	0,953	**0,985**
capsule	0,689	0,903	**0,994**	0,531	0,983	**0,990**
hazelnut	0,923	0,720	**1,000**	0,968	0,972	**0,988**
metal_nut	0,574	0,957	**0,997**	0,887	0,941	**0,987**
pill	0,603	0,824	**0,942**	0,887	0,944	**0,981**
screw	0,579	0,752	**0,964**	0,965	0,974	**0,989**
toothbrush	**0,958**	0,772	0,917	0,932	0,985	**0,989**
transistor	0,807	0,943	**0,995**	0,899	**0,976**	0,973
zipper	0,776	0,927	**0,977**	0,865	0,970	**0,981**

Bảng 3.4 cho thấy trung bình che đi nhiều khác biệt giữa các loại.

PatchCore cao nhất ở 13/15 loại theo image-AUROC và 14/15 loại theo pixel-AUROC. Hai loại thấp nhất là toothbrush (0,917) và pill (0,942). Toothbrush chỉ có 60 ảnh huấn luyện, ít nhất trong bộ dữ liệu, nên bộ nhớ ít mẫu bình thường hơn các loại khác.

PaDiM có pixel-AUROC cao (trên 0,9 ở mọi loại) nhưng image-AUROC thấp ở hazelnut (0,720), screw (0,752), toothbrush (0,772), pill (0,824), grid (0,829) và cable (0,838). Hazelnut là trường hợp rõ nhất: pixel-AUROC 0,972 cho thấy bản đồ vẫn khoanh đúng vùng lỗi, nhưng điểm ảnh lấy giá trị lớn nhất của bản đồ, nên chỉ cần vài vị trí phản hồi mạnh trên ảnh tốt là điểm của ảnh tốt bị đẩy lên. Hazelnut và screw cũng là hai loại vật thể xoay giữa các ảnh, phù hợp với nhận xét ở Chương 2 rằng thống kê theo vị trí bị ảnh hưởng khi vật thể không cố định. Tuy vậy, PaDiM và PatchCore khác nhau cả về cơ chế lẫn backbone (ResNet-18 so với Wide-ResNet-50-

2), nên thực nghiệm này không tách được phần chênh lệch do từng yếu tố.

Autoencoder có image-AUROC gần mức đoán ngẫu nhiên ở carpet (0,553), metal_nut (0,574), cable và screw (0,579), và pixel-AUROC gần ngẫu nhiên ở tile (0,559) và capsule (0,531). Ngược lại, nó đạt 0,958 ở toothbrush, cao nhất trong ba phương pháp, và trên 0,92 ở bottle, hazelnut, wood. Đây là các loại có nền đơn giản và lỗi lớn, dễ thấy.

Đối chiếu với ba giả thuyết đặt ra ở Mục 2.5:

(G1) **Đúng.** Theo image-AUROC, PatchCore cao hơn Autoencoder ở 14/15 loại và PaDiM ở 12/15 loại; theo pixel-AUROC, cả hai cao hơn Autoencoder ở đủ 15 loại.

(G2) **Đúng một phần.** Hai khoảng cách image-AUROC lớn nhất giữa PatchCore và PaDiM đúng là ở hai loại vật thể xoay: hazelnut (0,280) và screw (0,212). Nhưng metal_nut, cũng xoay, chỉ chênh 0,041, trong khi grid là bề mặt không xoay lại chênh 0,161. Như vậy ràng buộc vị trí giải thích được một phần; phần còn lại có thể do khác biệt backbone.

(G3) **Đúng.** Autoencoder tốt ở toothbrush, wood, bottle, hazelnut, nơi nền đơn giản và lỗi lớn; kém ở capsule và screw (lỗi nhỏ) và ở carpet, cable, metal_nut (nhiều chi tiết khó tái tạo).

3.4. Phân tích định tính

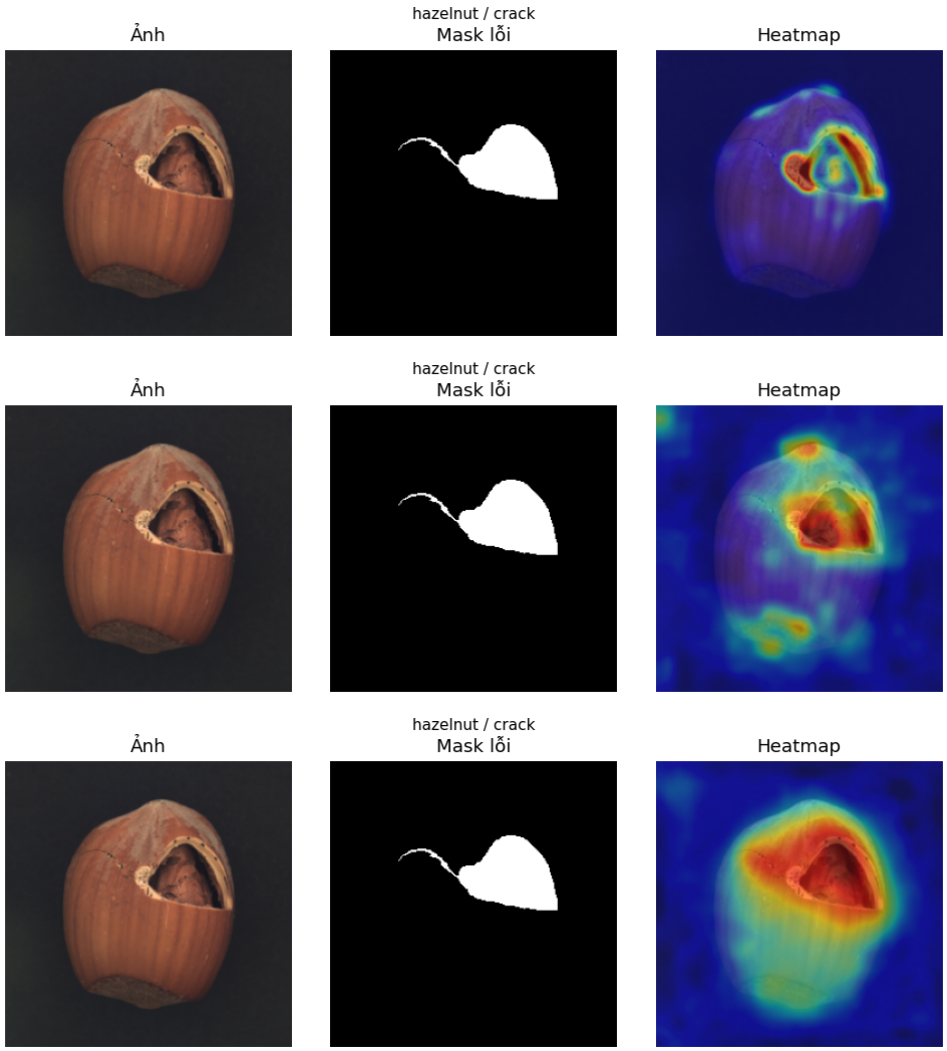
Hình 3.1 và Hình 3.2 so sánh bản đồ của ba phương pháp trên cùng một ảnh lỗi. Mỗi hàng gồm ảnh, mặt nạ lỗi và bản đồ bất thường; thang màu của mỗi phương pháp cố định cho cả loại đó (phân vị 1% và 99,5% của mọi bản đồ test), nên màu so sánh được giữa các ảnh cùng phương pháp nhưng không so sánh được giữa các phương pháp.

Với hạt dẻ nứt (Hình 3.1), Autoencoder chỉ sáng ở viền vết nứt, phần bên trong gần như không có phản hồi. Mạng dựng lại ảnh mờ, nên sai số tập trung ở nơi có biên sắc nét chứ không phủ hết vùng lỗi. PaDiM phủ được vùng nứt nhưng có thêm một vùng sáng ở đáy hạt, nơi không có lỗi; đây là kiểu phản hồi sai làm điểm ảnh tốt tăng lên như đã nêu ở mục trước. PatchCore phủ vùng nứt rõ nhất và nền gần như không có phản hồi.

Với viên nang có vết nứt nhỏ (Hình 3.2), Autoencoder phản hồi trên nền và dọc mép viên nang, không chạm vào vết nứt; điều này khớp với pixel-AUROC 0,531 của loại capsule. PaDiM và PatchCore đều có điểm sáng tại vết nứt. Cả hai còn phản hồi mạnh tại một vết nhỏ trên nền phía trên viên nang, có thể thấy trong ảnh gốc nhưng không được đánh dấu trong mặt nạ. Ví dụ này cho thấy một phần "báo động giả" của các phương pháp đặc trưng là những bất thường có thật nhưng nằm ngoài định nghĩa lỗi của bộ dữ liệu.

3.5. Chi phí tính toán

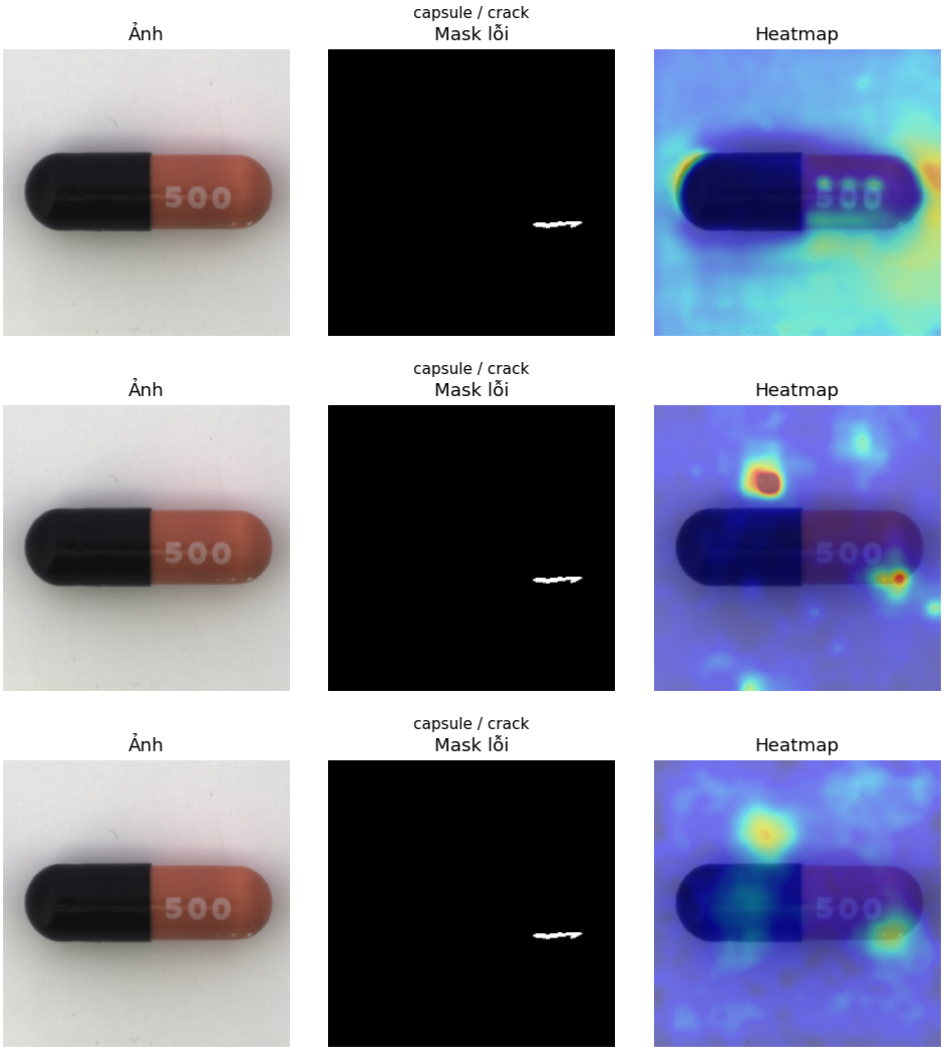
PaDiM nhanh nhất ở cả hai giai đoạn vì chỉ cần một lượt trích đặc trưng bằng ResNet-18 và phép nhân ma trận nhỏ tại mỗi vị trí. PatchCore chậm hơn PaDiM khoảng 5 lần khi xây dựng: thuật toán chọn coreset phải chọn tuần tự từng vector, và số vector tăng theo số ảnh, nên hazelnut (391



Hình 3.1. Hazelnut, lỗi nứt. Từ trên xuống: Autoencoder, PaDiM, PatchCore

Bảng 3.5. Chi phí tính toán trên Tesla T4

	Autoencoder	PaDiM	PatchCore
Thời gian xây dựng mô hình, cả 15 loại	22 phút	6 phút	30 phút
Thời gian suy luận trung bình (ms/ảnh)	44,8*	5,5	35,5
Dung lượng mô hình mỗi loại (MB)	45	158	36–235 (TB 145)



Hình 3.2. Capsule, lỗi nứt nhỏ. Từ trên xuống: Autoencoder, PaDiM, PatchCore

ảnh) mất 299 giây trong khi toothbrush (60 ảnh) mất 10 giây. Khi suy luận, PatchCore phải tìm láng giềng gần nhất trong bộ nhớ cho từng patch nên cũng chậm hơn PaDiM.

Có hai giới hạn khi đọc Bảng 3.5. Thứ nhất, *thời gian suy luận của Autoencoder có tính cả thời gian đọc ảnh từ Google Drive, còn của PaDiM và PatchCore chỉ tính lượt chạy mô hình, nên không so sánh trực tiếp được. Thứ hai, dung lượng của PaDiM và PatchCore chưa tính backbone học trước dùng chung (ResNet-18 khoảng 45 MB, Wide-ResNet-50-2 khoảng 265 MB).

3.6. So sánh với kết quả đã công bố

PatchCore với coreset 10% được báo cáo đạt image-AUROC 0,990 và pixel-AUROC 0,981 trên MVTec AD [3]; kết quả của nhóm là 0,983 và 0,980. Chênh lệch ở image-AUROC có thể do khác biệt tiền xử lý: bài gốc cắt vùng giữa 224×224 sau khi đổi kích thước, còn giao thức của nhóm giữ nguyên ảnh 256×256 để bản đồ khớp toàn bộ mặt nạ.

PaDiM được báo cáo đạt pixel-AUROC 0,975 với Wide-ResNet-50-2 [2]; nhóm dùng ResNet-18 và đạt 0,960. Với Autoencoder, bài báo PaDiM trích lại pixel-AUROC 0,82 cho Autoencoder dùng sai số bình phương [2]; Autoencoder của nhóm đạt 0,843. Kết quả của cả ba phương pháp nằm trong khoảng hợp lý so với các số đã công bố.

3.7. Hạn chế

- **Một lần chạy.** Chưa có độ lệch chuẩn qua nhiều seed. PaDiM và PatchCore gần như tất định: chạy lại loại bottle trên một GPU khác (RTX 3050) cho PaDiM cùng kết quả 0,9976/0,9776 và PatchCore lệch ở chữ số thứ tư. Autoencoder thì dao động: cùng mã nguồn, bottle đạt 0,926 trên T4 và 0,902 trên RTX 3050.
- **So sánh PaDiM và PatchCore không cùng backbone**, nên không tách được ảnh hưởng của cơ chế và của backbone.
- **Autoencoder chỉ thử một cấu hình** (sai số bình phương, latent 256). Khi chạy thử, mất mát có lúc tăng vọt (epoch 80 trên bottle) rồi hồi lại, cho thấy tốc độ học 10^{-3} có thể hơi cao.
- **Chưa chọn ngưỡng quyết định.** AUROC đánh giá khả năng xếp hạng; để dùng thực tế cần chọn ngưỡng từ ảnh tốt không tham gia huấn luyện và đo tỷ lệ báo động giả tại ngưỡng đó.

KẾT LUẬN

Đề tài đã xây dựng và đánh giá ba phương pháp phát hiện và định vị bất thường không giám sát trên đủ 15 loại của MVTec AD theo cùng một giao thức. Autoencoder do nhóm tự thiết kế và huấn luyện; PaDiM và PatchCore dùng đặc trưng học trước qua thư viện Anomalib.

Kết quả cho thấy PatchCore tốt nhất (image-AUROC 0,983, pixel-AUROC 0,980), sát số liệu của bài báo gốc. PaDiM đứng thứ hai (0,890 và 0,960), nhanh nhất, nhưng điểm ảnh kém tin cậy ở các loại vật thể xoay. Autoencoder thấp nhất (0,756 và 0,843): bản tái tạo mờ làm sai số xuất hiện ở viền và nền thay vì ở vùng lỗi, dù phương pháp vẫn tốt ở các loại có lỗi lớn như toothbrush. Trong phạm vi thực nghiệm này, dùng lại đặc trưng ImageNet hiệu quả hơn nhiều so với học tái tạo ảnh từ đầu trên vài trăm ảnh tốt.

Các hướng phát triển xuất phát trực tiếp từ hạn chế ở Chương 3:

- Chạy Autoencoder với nhiều seed và thử hàm mất mát SSIM [21] để biết phần nào của khoảng cách là do cấu hình.
- Chạy PaDiM và PatchCore với cùng backbone để tách ảnh hưởng của cơ chế khỏi ảnh hưởng của backbone.
- Chọn ngưỡng quyết định từ một tập ảnh tốt tách riêng và đo tỷ lệ báo động giả, làm cơ sở cho việc dùng mô hình trong một hệ thống kiểm tra thực tế.

TÀI LIỆU THAM KHẢO

[1] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600. DOI: 10.1109/CVPR.2019.00982 visited on Oct. 21, 2026. address: https://www.mvtec.com/fileadmin/Redaktion/mvtec.com/company/research/datasets/mvtec_ad.pdf

[2] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in *Pattern Recognition. ICPR International Workshops and Challenges*, ser. Lecture Notes in Computer Science, vol. 12664, Springer, 2021, pp. 475–489. DOI: 10.1007/978-3-030-68799-1_35 arXiv: 2011.08785. visited on Oct. 21, 2026. address: https://arxiv.org/abs/2011.08785

[3] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler, "Towards total recall in industrial anomaly detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14 318–14 328. arXiv: 2106.08265. visited on Oct. 21, 2026. address: https://arxiv.org/abs/2106.08265

[4] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. "Generalization," Dive into Deep Learning, visited on Oct. 21, 2026. address: https://d2l.ai/chapter_linear-regression/generalization.html

[5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. visited on Oct. 21, 2026. address: https://www.deeplearningbook.org/

[6] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola. "Multilayer perceptrons," Dive into Deep Learning, visited on Oct. 21, 2026. address: https://d2l.ai/chapter_multilayer-perceptrons/mlp.html

[7] PyTorch. "Optimizing model parameters," PyTorch Tutorials, visited on Oct. 21, 2026. address: https://docs.pytorch.org/tutorials/beginner/basics/optimization_tutorial.html

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015. address: https://arxiv.org/abs/1412.6980

[9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791

[10] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456. address: https://arxiv.org/abs/1502.03167

[11] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. address: https://arxiv.org/abs/1411.1792

[12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90

[13] S. Zagoruyko and N. Komodakis, “Wide residual networks,” in *Proceedings of the British Machine Vision Conference (BMVC)*, 2016. address: https://arxiv.org/abs/1605.07146

[14] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255. DOI: 10.1109/CVPR.2009.5206848

[15] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. DOI: 10.1016/j.patrec.2005.10.010

[16] S. Akcay, D. Ameln, A. Vaidya, B. Lakshmanan, N. Ahuja, and U. Genc. “Anomalib: A deep learning library for anomaly detection.” arXiv: 2202.08341. address: https://arxiv.org/abs/2202.08341

[17] Scikit-learn. “Scikit-learn: Machine learning in Python,” visited on Oct. 21, 2026. address: https://scikit-learn.org/stable/

[18] NumPy. “NumPy,” visited on Oct. 21, 2026. address: https://numpy.org/

[19] Matplotlib. “Matplotlib: Visualization with Python,” visited on Oct. 21, 2026. address: https://matplotlib.org/

[20] Scikit-learn. “sklearn.metrics.roc_auc_score,” Scikit-learn, visited on Oct. 21, 2026. address: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html

[21] P. Bergmann, S. Löwe, M. Fauser, D. Sattlegger, and C. Steger, “Improving unsupervised defect segmentation by applying structural similarity to autoencoders,” in *Proceedings of the 14th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP)*, 2019. address: https://arxiv.org/abs/1807.02011

[22] O. Sener and S. Savarese, “Active learning for convolutional neural networks: A core-set approach,” in *International Conference on Learning Representations (ICLR)*, 2018. address: https://arxiv.org/abs/1708.00489

[23] Nhóm 13, *Mã nguồn và tài liệu dự án MVTec anomaly detection*, 2026. visited on Oct. 21, 2026. address: https://github.com/AnDpTri/mvtec-anomaly-detection

PHỤ LỤC. MÃ NGUỒN VÀ CÁCH TÁI LẬP

Kho mã nguồn

Toàn bộ mã nguồn, cấu hình và tệp kết quả nằm trong kho mã nguồn của nhóm [23]:

https://github.com/AnDpTri/mvtec-anomaly-detection

Kết quả trong Chương 3 được tạo từ commit `a839dd0` (PaDiM, PatchCore) và `37fb3e5` (Autoencoder).

Thư mục	Nội dung
`configs/common.yaml`	Cấu hình chung: kích thước ảnh, chuẩn hóa, seed, danh sách 15 loại
`src/data/`	Đọc ảnh, mặt nạ và nhãn của MVTec AD
`src/evaluation/`	Tính AUROC, vẽ hình minh họa, ghi và tổng hợp kết quả
`src/autoencoder/`	Kiến trúc, huấn luyện và đánh giá Autoencoder
`src/padim/`, `src/patchcore/`	Chạy PaDiM và PatchCore
`src/common/`	Tiện ích chung, gồm lớp gọi mô hình của Anomalib
`notebooks/colab_run_all.ipynb`	Notebook chạy toàn bộ thực nghiệm trên Google Colab

Dữ liệu

Đường tải chính thức của MVTec AD không còn truy cập được khi thực hiện đề tài; nhóm dùng bản sao trên Hugging Face (`foersben/mvtec-ad`), cùng cấu trúc thư mục và số ảnh như Bảng 2.2. Lệnh `python tools/download_mvtec.py <thư mục>` tải về đủ 15 loại.

Chạy lại thực nghiệm

Từ thư mục gốc của kho mã nguồn, sau khi cài các thư viện trong `requirements.txt`:

```
python -m src.autoencoder.train --data-root <thư mục MVTecAD>
python -m src.padim.run         --data-root <thư mục MVTecAD>
python -m src.patchcore.run     --data-root <thư mục MVTecAD>
python -m src.evaluation.summary
```

Mỗi lệnh chạy đủ 15 loại, ghi một dòng vào `results/results.csv` sau mỗi loại và bỏ qua các loại đã có kết quả, nên có thể chạy tiếp sau khi bị gián đoạn. Lệnh cuối in bảng AUROC theo loại và trung bình theo nhóm như Bảng 3.3 và Bảng 3.4.